

Prediction-Driven Non-Preemptive Scheduling with Bounded Overtaking for Shared Execution Services under Deadline-Driven Bursty Load

[First Author Name]¹ and [Second Author Name]^{2,*}

¹ [First author: department, institution, city and postcode, country]; [first author e-mail]

² [Second author: department, institution, city and postcode, country]; [second author e-mail]

* Correspondence: [corresponding author e-mail]

Abstract

A shared execution service runs submitted jobs on a fixed pool of servers, without preemption, under a per-job time limit L . Reordering by predicted cost shortens most waits and can leave one job far behind. We prove a conservation law. For every non-preemptive work-conserving policy on k servers, every arrival sequence and every job, the delay relative to first-come first-served equals the net overtaken work divided by k , to within $(2 - 2/k)L$, an exact supremum. Bounding overtaken work is therefore sufficient and necessary for a per-job guarantee; counting overtakes is neither. A wrapper on that budget bounds every job's wait against first-come first-served under arbitrary arrivals and predictions. The bound reads the budget only through a cap, so the shape below the cap is free; the additive constant $(3 - 2/k)L$ is likewise an exact supremum, and the operator sets the cap from a promise in seconds. The score underneath should estimate expected cost, not a log-scale point prediction. We evaluate on two judging platforms, a serverless trace, two continuous-integration pools whose recorded waits check the simulator, and a compute farm. The method needs a time limit comparable to the waits at issue and heavy-tailed costs; we include a workload failing each.

Keywords: non-preemptive scheduling; scheduling with predictions; multiserver queues; bounded overtaking; per-job guarantee; first-come first-served; runtime prediction

1. Introduction

A shared execution service runs jobs on behalf of many users on a fixed pool of k servers. A job that has started is not interrupted: it runs until it finishes or until it reaches a per-job time limit L . Serverless function platforms, build and test farms for continuous integration, batch inference queues, and the judging back ends of programming-assignment platforms are all organised this way [1]. Operators document the consequence themselves: a contest judging system recommends one judging host per twenty teams and notes that with fewer hosts the waiting time grows [2], and the maintainers of a university auto-grader replayed the busiest four hours before a common deadline and found that one submission looping until its resource limit took a grader out of circulation and delayed the graders behind it [3]. Two properties of the offered load make such a service slow down when it is used most. Arrivals follow an external calendar, so they are neither stationary nor independent; the same concentration near a fixed cutoff has been measured directly [4–6]. And job costs are heavy-tailed: on the serverless invocation trace we use [7,8], the

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Mathematics* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](#) license.

longest 1% of invocations account for 44% of the total work, and on the judging traces, after the platform time limit is applied, the longest 1% still account for 37.5%. A few very long jobs therefore sit in front of many short ones exactly when the queue is longest.

The mechanism we exploit is visible in a queue of three jobs on one server (Figure 1). Job *A* loops forever and is stopped by the 60-second limit; jobs *B* and *C* take 0.2 seconds each. In arrival order the mean wait is 40.1 seconds; shortest first it is 0.2 seconds, at a cost of 0.4 seconds to *A*. The server performs the same total work in both orders. This is arithmetic rather than an experiment, and it rests on one assumption: that the scheduler knows, before *A* starts, that *A* will be long. That assumption raises three questions, and the paper is organised around them. Can the cost of a job be estimated at the moment it arrives, from history that is causally visible at that instant? Given estimates, what should a non-preemptive scheduler rank by? And what happens to an individual job when the estimates are wrong?

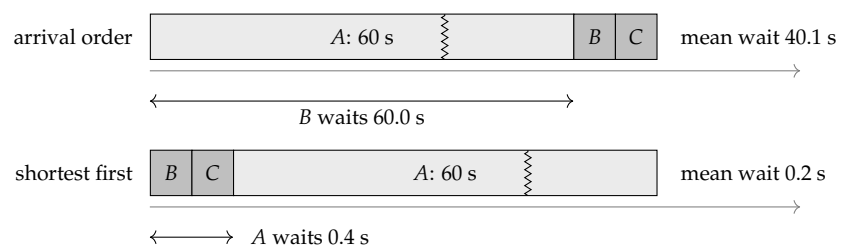


Figure 1. The three-job example of Section 1 on one server: *A* is stopped at the per-job limit $L = 60$ s, *B* and *C* take 0.2 s each, and all three are present at $t = 0$. The boxes are schematic, not to scale, and the zigzag marks the elided part of *A*. The two orders execute the same total work; reordering moves 60 seconds of waiting off *B* and *C* and puts 0.4 seconds onto *A*. Source: the arithmetic stated in the text, not a measurement.

On the first, arrival-time history is enough to tell the heavy jobs from the rest. Using only features whose values were already available at the arrival instant, gradient-boosted trees [9] reach an AUROC of 0.93, 0.93 and 0.98 for identifying heavy jobs on the three platforms we study. We compare three families of predictor under one visibility protocol and one split: tabular models, a sequence model over each user's history, and a heterogeneous graph neural network over the user, task and assignment entities [10]. The graph model does not improve on the tabular one here. An ablation that removes every edge, and then one that permutes the edges at random, leaves its classification scores where they were, which places the signal in each entity's own history rather than in the relational structure. We report this as a property of this prediction task, not as a general statement about relational models.

On the second question, the ranking score follows from non-preemption. Once a job starts, its whole realised cost is added to the wait of every job behind it, so waiting time is additive in realised cost and jobs should be ordered by the conditional expectation $\mathbb{E}[C \mid x]$. A point prediction fitted with squared error on a log scale estimates a conditional geometric mean instead; the two differ by a conditional variance term, and that term is concentrated on the same small fraction of heavy jobs that the schedule is trying to move. The errors are also asymmetric. Replacing the worst 1% of under-estimates by true costs closes 0.91 of the remaining gap to a scheduler that knows true costs, while replacing the worst 1% of over-estimates closes about 0.25. Putting a heavy job early is the expensive mistake.

The third question decides whether an operator would run any of this. Ordering by prediction improves the bulk of the distribution and can damage the tail: in our busiest configuration one job waited 5,905 seconds longer than it would have under first-come first-served. The robustness results we are aware of in the scheduling-with-predictions

literature bound a mean relative to an optimal or shortest-remaining-processing-time reference, and the constant-factor ones require the prediction error to be bounded multiplicatively, which is not a property anyone can promise about a deployed predictor [11,12]. We change the reference point instead of strengthening the assumption, and bound, for each job separately, how much longer it waits than it would have waited under first-come first-served on the same arrival sequence, assuming nothing about the arrival process and nothing about prediction quality. The mechanism is an overtake budget: for every waiting job we account the total work of later-ordered jobs that have completed while it waited, and when that account reaches a budget the job is dispatched next. The guard wraps an arbitrary base scheduler, including a learned one, and touches only the dispatch decision. It is one mechanism with one parameter that matters for the promise, a cap $B_{\max}$ on the budget; what the budget does below that cap is a design freedom that changes how much of the base policy's benefit survives and which jobs pay for it, and changes the promise not at all.

Budgeting work rather than counting overtakes is forced rather than chosen. We prove an identity that holds for every non-preemptive, work-conserving k -server policy and every arrival sequence. A job's delay relative to first-come first-served equals its net overtaken work divided by k , to within $(2 - 2/k)L$, with equality when $k = 1$. That constant is the exact supremum of the error, approached by an explicit family and never attained, and its two directions are not alike: upward the victim really does wait $(2 - 2/k)L$ longer than its net overtaken work predicts, while downward about half of the extremal value is the bookkeeping convention that charges an overtaker dispatched in the victim's own phase, which executes nothing while the victim waits, and there the victim's own excess is negative. The identity does not refer to the policy, and it cuts both ways. Bounding overtaken work is sufficient for a per-job guarantee, and any policy that offers such a guarantee must bound it, so mechanisms that limit the number of overtakes are neither sufficient nor necessary for it. The guarantee is denominated in units of L : it says something when L is comparable to the waits an operator cares about and nothing when it is not, and a computing farm whose jobs run for days is a case where the method should not be used, which we include.

We make four contributions.

- The net-overtake identity just stated, with the constant $(2 - 2/k)L$ shown to be an exact supremum, approached and never attained, and a corollary in the other direction: a bound on overtaken work is necessary for any per-job guarantee of this kind.
- An overtake-budget guard. It wraps any base scheduler and bounds every job's wait against first-come first-served, for k servers, arbitrary arrivals and arbitrary predictions. The bound reads the budget only through a cap: any budget rule with $0 \leq \text{budget} \leq B_{\max}$ gives the same guarantee, so the operator sets $B_{\max} = k(G - (3 - 2/k)L)$ from a promise G in seconds, with no data, and is then free in how the budget is spent below it. The additive constant $(3 - 2/k)L$, beyond the budget term, is an exact supremum, never attained, and it grows with k from L at $k = 1$ towards $3L$; no analysis of this mechanism can improve it. One shape of budget, the one whose allowance grows with the wait a job has already had, carries a multiplicative bound as well. We state which assumptions can be dropped, and show by counterexample that work conservation cannot.
- A comparison of cost predictors on causally visible history, covering tabular, sequence and heterogeneous graph models, with two conclusions that carry into the scheduler: rank by expected cost rather than by a point or log-scale prediction, and treat the error as asymmetric.

- An evaluation on real logs from two programming-assignment judging platforms [13,14] and a serverless invocation trace [7], with every simulated job checked against the theoretical bound, together with workloads on which the method does not apply [15,16] and a statement of the boundary.

All figures quoted above come from development data. The sealed part of each trace is opened once, after the method, the parameters and the main metric have been frozen, and the final numbers are reported separately from these.

Section 2 places the paper in the literature and Section 3 states the model, the visibility protocol and the metrics. Sections 4 and 5 give the cost predictors and the scheduler, Section 6 the identity, the per-job bounds and their tightness, and Section 7 the traces. Section 8 reports the experiments, Section 9 the limitations, and Appendix A the tightness constructions.

2. Related Work

2.1. Scheduling with predictions

Purohit, Svitkina and Kumar [17] frame predictions in online algorithms as a trade-off between consistency and robustness, and Mitzenmacher [18] analyses a non-preemptive priority queue whose classes come from predicted sizes; our argument for ranking by expected cost is a sample-path counterpart of the latter. Scully, Grosz and Mitzenmacher [11] assume, in a preemptive M/G/1, that the estimate of a job of size s lies in $[\beta s, \alpha s]$, show that feeding estimates to SRPT is not enough on its own, and give a policy within α/β of SRPT in expected response time. Worst-case analyses give ratios for total flow time that degrade with the distortion of the predictions instead of requiring it in advance [19,20]; others cover noisy estimates in a many-server limit [21] and the cost of obtaining a prediction [22]. The survey of Mitzenmacher and Shahout [12] leaves open what a system should do when a predictor degrades. Our guarantee differs from all of this on four axes at once: the reference is first-come first-served rather than an optimal or SRPT schedule, the statement is per job rather than about a mean, the assumptions are an arbitrary arrival sequence and arbitrary predictions rather than Poisson arrivals with bounded multiplicative error, and service is non-preemptive. That combination has a cost. A bound relative to first-come first-served is weaker than one relative to an optimum, since first-come first-served is itself poor under heavy-tailed costs; it keeps a job from a catastrophic wait and claims no improvement by itself.

2.2. Fairness relative to FCFS, and bounded overtaking

Comparing a job against the schedule it would have received under first-come first-served is established in parallel job scheduling: Sabin and Sadayappan [23] define a job's fair start time as its start time in a run with no backfilling and no later arrivals, and measure unfairness as the excess over it. That is the quantity we bound, and there it is measured after the fact; we are not aware of a scheduler that enforces it online, or of an upper bound proved for it. Backfilling gives guarantees of another kind, a reservation to every job in the conservative variant and to the queue head in the aggressive one, which rules out starvation without bounding anything for the rest [24,25]. Bounded overtaking already has a meaning in queueing theory, and it is a count. Nudge [26] lets a small arriving job swap past the job at the tail of the queue, Nudge- K [27] lets it pass up to K others while no job is passed more than once, and the line continues with Nudge*(M) [28]; these are analysed in steady state under Poisson arrivals with light-tailed or phase-type sizes, and yield, for suitable parameters, stochastic dominance over first-come first-served. Because a job is passed at most once, Nudge already implies on every sample path a per-job additive bound of one small-job length, so we do not claim that form of result as new. The

finite-skip models of WCFS restrict service to the oldest n jobs [29], and the same positional notion is a proof device in fluid-limit analyses of inference serving [30]. Additive per-individual bounds against a reference discipline are older still [31,32], while aging disciplines [33,34] and the unfairness analyses of SRPT against processor sharing [35,36] describe classes in steady state rather than individual jobs. What differs here is the budgeted quantity: we limit overtaking work rather than the number of overtakes, enforce it online under arbitrary predictions, and show the choice is forced, since bounding work is sufficient and necessary for a per-job guarantee while counting positions is neither. A single-server argument also stops working with several servers, since the workload vector under first-come first-served is minimal, in the sense of weak majorization, among service orders [37,38]; our identity replaces that step with a quantitative comparison.

2.3. Runtime prediction and scheduling in deployed systems

Backfilling schedulers on high-performance computing systems have used predicted runtimes in place of user estimates for two decades [39], with predictions learned from past jobs and queue reordering tuned online [40,41]; their resources are multi-dimensional and their reported metric is a mean bounded slowdown. Closest to our application setting, Zhang, Wu and Lu [42] predict the running time of submitted code in a judging queue, cut the queue into fixed-size windows and dispatch in ascending predicted runtime within each window, reporting mean response time against first-come first-served. We are therefore not the first to schedule a judging queue by predicted cost. A fixed window does limit overtaking, but by position, and that work states no bound, tests no adversarial predictions and does not evaluate on real arrival times. In inference serving, where non-preemption is close to a hard constraint, recent systems learn a relative order instead of a length [43] or target the tail because prediction-driven policies are fragile under distribution shift [44,45], and production systems there avoid starvation with aging counters and maximum waiting times, without proofs. Our guard is a wrapper rather than a policy, so it can be placed around any of these schedulers, and its bound does not depend on which one is underneath.

2.4. Time-varying and deadline-driven load

That people concentrate their actions near a cutoff has been measured directly, in time management research [4] and in conference registration data [5,6]. The queueing consequences of an arrival rate that varies on the timescale of the service time are also well studied: peak congestion in a multi-server system lags the peak of the arrival rate [46], and staffing rules that follow a time-varying rate rest on offered-load approximations rather than stationary formulas [47–49]. These results say how many servers to provide. They give no waiting bound for an individual job and no guidance for a queue that has already formed, and we leave capacity fixed. The steady-state classified-priority formula that motivates ranking by predicted cost [18,50] does not survive contact with this load either; Section 3 measures the mismatch, which is why we state the guarantee on sample paths and assume nothing about arrivals.

2.5. Relational and tabular models for cost prediction

Our comparison includes a heterogeneous graph neural network over users, tasks and assignments, in the relational graph convolution family [10,51]. The published benchmark evidence needs care before it is used in either direction. In RelBench [52] and its successor [53], the gradient-boosted baseline is fitted on the raw columns of the entity table alone, without hand-built aggregates over related tables, so the margin reported there is not a comparison against a tabular model carrying engineered history features. The same paper's study against a human data scientist is the closer comparison: the relational

model matched or beat the expert on 11 of 15 tasks, while the authors record that regression tasks are where it struggles. Cost prediction is a regression task on entities with long individual histories, which places it in that region, and the evidence that tree ensembles remain strong on tabular data concerns single tables [54]. We therefore settle the question inside our own protocol instead of importing a conclusion, with a tabular baseline that carries engineered causal history features [9], an ablation ladder that removes and then permutes the edges, and a comparison made on queueing outcomes rather than on AUROC alone. The negative result we report is about this task on these traces, where each user and each task already has enough history of its own; the only relational signal we find is on task types that have none yet.

3. Problem Model

3.1. Jobs, Servers and Dispatch

A finite set of n jobs is served by k identical servers. We use *server* throughout for what a deployment would call an executor, a judge host or a worker, and *job* for what a platform would call a submission, an invocation or a run. Job i arrives at time $a_i \geq 0$ and requires C_i units of service. Service is non-preemptive, so a server that starts a job keeps it until the job finishes or until it has been running for the per-job time limit L , at which point the system stops it. Throughout, C_i denotes executed work, so $0 < C_i \leq L$ by construction and a job stopped at the limit contributes exactly L .

Servers are work-conserving, meaning that no server idles at an instant at which some job waits. Jobs carry a rank, the total order induced by the pair (arrival time, input index); we write $j < i$ for “ j has the smaller rank”, which implies $a_j \leq a_i$. Events occurring at a common instant are processed as completions, then arrivals, then dispatches. A policy is any rule that, whenever a server is free and at least one job waits, names one waiting job and dispatches it; it may be randomised, learned, priority-driven, or fed by predictions of arbitrary quality, and we assume nothing else about it.

Arrival times are arbitrary. We place no stochastic assumption on a_i , on C_i , or on the dependence between them, because the load we are concerned with is generated by an external calendar and is neither stationary nor Poisson.

3.2. What the Scheduler Knows

The service time C_i is hidden at arrival. The scheduler reads it at the instant i completes and not before, which is the situation of any service that runs submitted work without inspecting it first. Available at a_i are a feature vector x_i and a score $\hat{C}_i$ computed from it.

Visibility is causal, and we define it by result availability rather than by timestamp order. Write done_j for the instant at which the outcome of job j became readable and $\delta \geq 0$ for the delay with which outcomes reach the feature store. Then

$$\text{information about } j \text{ may enter } x_i \iff \text{done}_j + \delta \leq a_i.$$

Every historical feature is recomputed under this rule rather than assembled once and masked afterwards; Section 4 gives the procedure and the sensitivity to δ .

3.3. Waiting Time, the FCFS Reference and the Service Guarantee

Let s_i^P be the start time of job i under policy P and $W_P[i] = s_i^P - a_i$ its waiting time, which excludes service. FCFS is the policy that always dispatches the waiting job of smallest rank, and that dispatches in increasing rank when several servers become free at the same instant. We run it on the same arrival sequence, with the same service times and the

same k , so that the comparison is pathwise rather than distributional. The per-job excess is

$$\text{excess}_P[i] = W_P[i] - W_{\text{FCFS}}[i].$$

Two further quantities carry the analysis. Writing $j \prec i$ for “ j is dispatched before i ” in the dispatch sequence,

$$\text{In}_i = \sum_{j>i, j\prec i} C_j, \quad \text{Out}_i = \sum_{j<i, i\prec j} C_j,$$

the work that overtook i and the work that i overtook. Section 6 relates them to $\text{excess}_P[i]$.

We say that a policy P offers a service guarantee G if $\text{excess}_P[i] \leq G$ for every job i , on every arrival sequence, for every vector of service times bounded by L , and for every score, adversarial scores included. The statement is per job and holds on the realised sample path rather than in distribution or on average, and it does not presume that the score is any good. A per-individual additive bound relative to a reference discipline is familiar from packet scheduling [31], and the same difference has been measured after the fact as a fairness diagnostic in batch schedulers [23]. The robustness results in the scheduling-with-predictions literature are of a different kind, comparing against an optimal policy in an average sense under a bound on the prediction error [11,17].

We take FCFS as the reference because it is what these services deploy by default, so the excess is a quantity an operator can be held to. It also has known extremal properties for the unfinished-work vector in a multiserver system [38], which is why single-server arguments do not transfer unchanged to $k > 1$.

3.4. Metrics

The primary metric is the 99th percentile of waiting time inside deadline windows. A deadline window is the 24 hours preceding a deadline; a job belongs to the window if and only if its arrival falls inside it, and a job that arrives inside but completes after the window closes is still counted. We fix this choice before any sealed part of a trace is opened. We also report four secondary quantities. The mean wait over all jobs. The maximum of $\text{excess}_P[i]$, which is what a service guarantee bounds. The *harm*, the largest excess over FCFS among the jobs FCFS would have started within 1 s, that is, among the jobs FCFS would not have delayed at all; a guarantee G says nothing about how that excess is distributed, and the harm is where an operator feels it. The *firing rate*, the share of dispatch epochs at which the guard of Section 5 overrides the base policy, weighted by queue length. We also report the worst wait among heavy jobs, heavy being the label Section 4 defines.

Improvements are also expressed as the fraction of the gap closed,

$$\frac{\overline{W}_{\text{FCFS}} - \overline{W}_P}{\overline{W}_{\text{FCFS}} - \overline{W}_{\text{SJF}}},$$

where SJF sorts waiting jobs by their true service times. SJF on true sizes is a reference policy and not an optimum. It is not deployable, since C_i is unknown at arrival, and it does not minimise any of the metrics above on a non-preemptive multiserver system. The ratio is unstable when its denominator is small, so we mark such configurations and print the two absolute waits next to every ratio. The fraction of the gap closed is not a percentage reduction in waiting time and we do not report it as one.

3.5. Why the Design Is Not Based on a Steady-State Model

Stationary queueing models give closed forms for the mean wait of a non-preemptive priority class [50] and would let class thresholds be placed analytically. They need a sta-

tionary arrival process, which deadline-driven load does not supply. Rather than assert that the mismatch matters, we measured it. On a synthetic Poisson stream whose service distribution matches the first two moments of the real one, the formula agrees with simulation to 0.4%. On the same workload replayed at its recorded timestamps, the measured wait is 10.7–13.7 times what the formula predicts on the development trace. A threshold tuned on the formula would be tuned against a number that is off by an order of magnitude.

We therefore use steady-state expressions as motivation only, and every promise made in this paper is a per-job statement on the realised sample path (Section 6). The mechanism that makes reordering worth doing is the three-job arithmetic of the introduction, where total work is unchanged while one long job moves back a little and several short ones move forward a lot.

4. Cost Prediction

The scheduler of Section 5 needs a number at the instant a job arrives. This section says what that number estimates, how we keep its inputs causally visible, which model families we compared under that one protocol, and which model we send into the queue. All figures are development figures, and the traces they come from are described in Section 7.

4.1. Two Targets

A scheduler and an evaluation want different things from the same model. Waiting time is additive in realised service, so the quantity that belongs in a ranking score is the conditional mean $\mathbb{E}[C_i \mid x_i]$, and Section 5.1 fits it directly. For comparing predictors we also need a target that stays interpretable when the mix of jobs shifts between periods, so we score each model on a binary label. A job is *heavy* when its executed cost C_i exceeds the 95th percentile of the training rows; this is the one definition of heavy used in the paper, and the worst heavy-job wait of Section 3 is taken over the jobs it selects. On the main judging trace that cut-off is 1.559 s and heavy jobs are 1.31% of the test period; on the second judging trace it is 1,314 ms and 5.07%. The label is deliberately fixed on training rows and never refit on the period being scored, which is why its rate in the test period is a quarter of its rate in training. Because the positive class is small we report average precision next to AUROC, and we report RMSE on $\log(1 + C)$ and Spearman correlation for the regression output, since a ranking policy consumes order rather than level.

4.2. The Visibility Protocol, Made Operational

Section 3 defines visibility by result availability: information about job j may enter x_i exactly when $\text{done}_j + \delta \leq a_i$. Turning that into a feature pipeline takes three commitments.

First, every historical feature is recomputed by streaming the log in availability order and reading each entity's state as of the query instant, rather than being assembled over the whole period and masked afterwards. We test this the expensive way: on 3,000 randomly drawn development jobs all 34 history and relational features are recomputed from scratch under each configuration's own rule and compared against the pipeline's values, and 0 rows disagree in any of the eight configurations. The same test applied to the natural but wrong construction, in which a feature may use any record with a smaller index, fails on 97.67% of rows under the primary rule. The graph pipeline is checked separately: an index assertion over 863,149 rows finds 0 timeline entries pointing at or after the read, brute-force recomputation on 600 rows matches on all 2,400 root states and all 2,400 neighbour sets, and perturbing or deleting every record after a cut time leaves all earlier predictions unchanged while a positive control moves the later ones.

Second, thresholds are refit per target period. The heavy cut-off and the class boundaries used inside the models are recomputed from the training rows of that target, and each target period is predicted by a model fitted only on periods before it, frozen before the target's first arrival. Training rows whose results would not have been available by that instant are dropped; on the primary configuration the rule binds on 0 rows, which is worth reporting because a rule that never binds is easy to believe is absent.

Third, we treat the delay δ with which results reach the feature store as a sensitivity rather than an assumption. At $\delta = 0$ the full tabular model reaches AUROC 0.9283; at $\delta = 600$ s it reaches 0.9066 and its average precision falls from 0.289 to 0.139. On the second judging platform, where the log carries no clock and lag has to be expressed in records, the same model moves from 0.9266 at lag 0 to 0.8902 at lag 1,000. Ranking survives a ten-minute feedback delay; the probability calibration does not survive it as well.

4.3. Predictor Families

All families see the same rows, the same split and the same availability rule. The tabular family is gradient-boosted trees [9] over five feature groups: task history (M1), user history (M2), static features of the submitted artefact (M3), everything tabular (M4), and M4 plus one-hop relational aggregates over the task-assignment and user-cohort neighbourhoods (M5). A control (M6) permutes task identity across assignments. The sequence family is a GRU over the earlier jobs of one user whose results are already available, used alone (R1) and as a state vector appended to M4's columns inside the boosted model (R1S). The relational family is a heterogeneous network over user, task, assignment and cohort nodes [10] with no node-identity embeddings, so an unseen task or user is handled by the same weights. Its ablation ladder runs N0 (a multilayer perceptron on exactly M4's columns), G0U (the relational architecture with every neighbour slot empty and the assignment and cohort roots removed, which leaves exactly M4's information), G0 (empty neighbour slots, roots kept), G1 (the real edges) and G2 (the same network on edges permuted within a period and time block, so that neighbour counts and recency are preserved and only the identity of the neighbour changes).

4.4. What the Comparison Shows

Table 1 gives the two judging traces. One-hop relational aggregates add nothing on either: M5 – M4 is +0.0019 of AUROC on the first and –0.0002 on the second. The ladder tells the same story with a trained model in place of hand-built aggregates. Deleting every edge (G0) and permuting the edges (G2) leave AUROC within 0.002 of G1, and the paired intervals for G1 – G0 (–0.0001 [–0.0019, 0.0018]) and G1 – G2 (+0.0019 [–0.0000, 0.0042]) cover zero. The one step of the ladder that resolves is N0 → G0U, +0.0139 [+0.0090, +0.0195] of AUROC at identical information, which is a statement about the encoder and not about edges.

Edges do carry something, and it is confined to cold-start tasks. On the 1,092 test rows whose task has fewer than five visible prior results, G1 beats M4 by +0.1464 [+0.0997, +0.1887] of Spearman correlation, and beats its own permuted-edge control by +0.0282 [+0.0130, +0.0432]. Over all rows the G1 – G2 Spearman difference is +0.0063 [+0.0014, +0.0111], small and resolved; the corresponding AUROC and AUPRC differences are not. We read this as a task property rather than a verdict on relational models: when each entity's own recent record is visible and rich, a neighbourhood adds little, and the place where it adds anything is the entity that has no record yet.

The sequence state is the strongest predictor we fitted and does not translate. R1S reaches AUROC 0.9366 against M4's 0.9283, a difference of +0.0083 [+0.0038, +0.0131], and it also wins on RMSE and Spearman. Inside the queue it is not a consistent improve-

ment: against the same ranking objective it gains +0.007 [0.003,0.011] of the deadline-window gap at busy-hour utilisation 0.8, loses −0.010 [−0.022, −0.002] at 0.5, and is unresolved at 1.0. A control fitted on the same 90% of rows that the GRU’s early stopping left available moves the same way, so the smaller training set does not explain it.

Which information matters is not a constant across platforms. On the first judging trace the user’s own history and the static features of the artefact beat the task’s history; on the second the task’s history beats the user’s by 0.11 of AUROC (Table 1). A task on the first platform is attempted by one cohort within a few days, so who is writing still carries information; a task on the second is attempted by thousands of users over years, so the task’s own record has already fixed the cost. Table 2 pushes this further on the non-judging traces. On the serverless trace the identity of the function explains 0.867 of the variance of $\log(1 + C)$ on its own, and static and calendar features alone are close to useless for ranking (Spearman 0.021). The two CI pools sit at opposite ends, entity history giving AUROC 0.9968 on one and 0.6334 on the other, because that pool’s job labels do not separate its costs; and the compute-farm trace is predictable in rank and not in level.

4.5. Cost of Predicting, and What Enters the Scheduler

Prediction has to be cheap enough to sit in front of a queue whose median job is 204 ms. Measured in a streaming loop over 20,000 jobs, the tabular path costs a median of 0.076 ms per job and a 99th percentile of 0.188 ms; the relational network costs 0.434 ms per row plus 0.121 ms of graph lookup, and the GRU 0.576 ms. Charging the measured feature and inference latency to the predictive policies before they enqueue changes the closed gap by less than 0.001 at every load we ran, so the choice between these predictors is not a latency decision at this job size.

It is a queueing decision. M5 has the higher AUROC of the two tabular candidates and loses in the queue at all three loads, by 0.012, 0.008 and 0.010 of the deadline-window gap, with intervals excluding zero in each case. R1S has the higher AUROC of everything we fitted and does not win consistently. We therefore take M4, the full tabular model, into Section 5, and we say plainly why: it was selected on the queueing outcome on validation periods, not on prediction quality, and the two orderings do not agree.

Table 1. Predictors on the two judging traces under one visibility protocol and one split. Heavy is the training-period 95th percentile of executed cost. RMSE and Spearman (Sp.) are on $\log(1 + C)$. R-net is the heterogeneous relational network.

Model	Information	AUROC	AUPRC	RMSE	Sp.
<i>CodeBench</i> , $n = 40,844$					
M1	task history	0.7960	0.053	0.3000	0.219
M2	user history	0.8591	0.242	0.2605	0.302
M3	static features	0.8632	0.082	0.2892	0.290
M4	all tabular	0.9283	0.289	0.2519	0.312
M5	M4 + one-hop	0.9302	0.299	0.2507	0.344
N0	MLP on M4’s columns	0.9209	0.306	0.2463	0.278
G0U	R-net, no edges or roots	0.9348	0.322	0.2417	0.321
G0	R-net, no edges	0.9325	0.316	0.2402	0.324
G1	R-net, real edges	0.9324	0.322	0.2397	0.329
G2	R-net, permuted edges	0.9305	0.318	0.2396	0.323
R1	GRU over user history	0.9239	0.324	0.2396	0.313
R1S	M4 columns + GRU state	0.9366	0.315	0.2414	0.329
<i>ACcoding</i> , $n = 647,465$; the neural models were not fitted here					
M1	task history	0.8990	0.431	0.2904	0.683
M2	user history	0.7855	0.292	0.3190	0.471
M3	static features	0.7884	0.165	0.3429	0.493
M4	all tabular	0.9266	0.527	0.2714	0.692
M5	M4 + one-hop	0.9264	0.527	0.2713	0.696

Table 2. The same protocol on the non-judging traces, by feature group. ENT uses the entity's own visible history, STATIC uses only what is declared before the job runs, ALL is both. RMSE and Spearman are on $\log(1 + C)$.

Trace	Features	AUROC	AUPRC	RMSE	Spearman
Serverless invocations	ENT	0.9828	0.816	0.3401	0.953
	STATIC	0.6238	0.084	0.8088	0.021
	ALL	0.9847	0.830	0.3185	0.932
CI pool, hardware A	ENT	0.6334	0.157	0.3964	0.833
	STATIC	0.4990	0.045	0.7659	0.277
	ALL	0.6512	0.177	0.3956	0.837
CI pool, hardware B	ENT	0.9968	0.984	0.0887	0.985
	STATIC	0.6915	0.077	0.6411	0.500
	ALL	0.9970	0.985	0.0860	0.988
Compute farm	ENT	0.8903	0.265	1.4214	0.883
	STATIC	0.8704	0.242	2.2559	0.750
	ALL	0.8902	0.260	1.3485	0.892

5. Scheduling Algorithm

The scheduler has two layers. A base policy orders the queue by a score computed at arrival, and a wrapper limits how much work may overtake any waiting job. The two are independent, in that the wrapper accepts any base policy and its guarantee does not depend on that policy behaving well.

5.1. Ranking by Expected Cost

When a server frees, the base policy takes the waiting job with the smallest score. Since service is non-preemptive, a dispatched job holds its server for its entire service time, and the delay it imposes on every job still queued is C_i itself rather than a monotone transform of C_i . Waiting times add in the units in which service is measured, so the score should estimate the conditional mean $\mathbb{E}[C_i \mid x_i]$.

This is not what a log-scale regression produces. Fitting squared error on $\log(1 + C)$ and inverting gives a conditional geometric mean, which differs from the conditional mean by a term that grows with the conditional variance, and in a heavy-tailed cost distribution that term sits on the small fraction of jobs the scheduler most needs to push back. We change the training objective and nothing else, keeping the features, the model size and the rolling-origin protocol of Section 4 as they are, with the target taken in raw seconds under a Tweedie loss of power 1.5, which accommodates a distribution with most of its mass near zero and a long right tail [55,56]. We write SPJF-E for the policy that ranks the queue by this estimate of expected cost, and SPJF-log for the same pipeline fitted on $\log(1 + C)$ and inverted.

Only the order of the fitted score is used. Its level does not transfer across periods, since the share of heavy jobs in the training periods differs from the target period by a factor of several, so we never read the score as a quantity, neither as an estimated finish time nor as a threshold. The scheduler is therefore invariant to a monotone recalibration of the score.

Section 8.3 measures what the change of estimand is worth, together with the alternatives we tried and rejected: a predicted quantile, a two-class rule, and squared error on raw seconds. It also measures the asymmetry of the errors, which is what makes the estimand matter: treating a heavy job as light and putting it in front costs about an order of magnitude more than the reverse mistake, so a score fitted for symmetric accuracy optimises the wrong thing.

Changing the score does not change the worst case. A job the score misjudges can still be passed indefinitely, and no choice of ranking function repairs that; Section 5.2 is what bounds it.

5.2. The Overtake-Budget Guard

Fix a base policy A . For every waiting job q and every time t define

$$\text{over}[q](t) = \sum \{ C_j : j > q \text{ in rank, } j \text{ has completed by } t \},$$

the total true service of the higher-ranked jobs that have finished by t . Such a job arrived at or after a_q , so it necessarily ran while q waited. Job q is allowed a budget $\text{budget}(q, t)$ on that quantity, and the fired set at a dispatch epoch is $E(t) = \{ q \text{ waiting} : \text{over}[q](t) \geq \text{budget}(q, t) \}$. At every dispatch epoch, that is, at every instant at which a server is free and the queue is non-empty, the wrapper serves the minimum-rank member of $E(t)$ when $E(t)$ is non-empty and otherwise serves whatever A chooses (Figure 2).

Serving the minimum-rank member matters when budgets differ between jobs. It guarantees that whenever a job j is dispatched ahead of a lower-ranked waiting job i , the job i was not in $E(t)$, hence $\text{over}[i](t) < \text{budget}(i, t)$ at that very instant, which is the step the per-job bound of Section 6 rests on. With a constant budget $\text{budget}(q, t) \equiv B$ the quantity $\text{over}[q](t)$ is non-increasing in rank, so $E(t)$ is non-empty exactly when the queue head belongs to it and the rule degenerates to a test on the head. Setting $B = 0$ recovers FCFS.

One mechanism, one cap, and a free choice of shape.

The guarantee of Section 6 reads the budget only through an upper bound on it: if $0 \leq \text{budget}(q, t) \leq B_{\max}$ for every waiting job and every instant, then every job waits at most $W_{\text{FCFS}} + B_{\max}/k + (3 - 2/k)L$, for any base policy, any arrival sequence and any predictions. Everything below the cap is a design freedom that changes how much of the base policy's benefit survives and which jobs pay for it, and changes the promise not at all. We study the one-line family

$$\text{budget}(q, t) = \min(B_0 + \gamma n_q + \eta k(t - a_q), B_{\max}),$$

with $B_0, \gamma \geq 0$, $0 \leq \eta < 1$ and n_q the number of jobs waiting when q arrives, and read three shapes off it: *constant* ($\gamma = \eta = 0$), *age-relative* ($\eta > 0$), and *queue-length* ($\gamma > 0$). The coefficient η says that a waiting job tolerates at most a fraction η of the system's entire service capacity, over its own waiting time, being spent on jobs that arrived after it, so the allowance grows with the wait a job has already had; being a fraction of capacity it is dimensionless and needs no retuning when k or the load changes. The term γn_q instead widens the allowance only for jobs that arrive into a queue that is already long, and is flat in the wait. B_0 , γ and $B_{\max}$ are measured in work and scale with k .

Only the age-relative shape carries a second, multiplicative bound, $W \leq (W_{\text{FCFS}} + B_0/k + (3 - 2/k)L)/(1 - \eta)$, and only when $\gamma = 0$: with $\gamma > 0$ the constant part of the budget differs from job to job, so the multiplicative form holds per job with that job's own constant rather than with a number the operator can state in advance (Remark 3). For $\gamma > 0$ we claim the additive bound alone.

Parameters from a service promise.

The operator states G , the number of seconds by which any job may exceed its FCFS waiting time, and reads off $B_{\max} = k(G - (3 - 2/k)L)$, which is the inversion of the bound proved in Section 6, and no data are needed for this step. The rule requires $G >$

Algorithm 1 Overtake-budget guard wrapped around a base policy A

```

1: parameters: cap  $B_{\max} \geq 0$ ; shape  $B_0, \gamma \geq 0, \eta \in [0, 1]$ ; base policy  $A$ 
2: state: waiting set  $Q \leftarrow \emptyset$ ;  $\text{over}[q] \leftarrow 0$  for every job  $q$ 
3: while some job has not started do
4:    $t \leftarrow$  next event time
5:   for all jobs  $c$  completing at  $t$  do  $\triangleright C_c$  becomes known here and nowhere earlier
6:     for all  $q \in Q$  with  $q < c$  in rank do
7:        $\text{over}[q] \leftarrow \text{over}[q] + C_c$ 
8:     end for
9:   end for
10:  for all jobs  $i$  arriving at  $t$  do
11:     $n_i \leftarrow |Q|$ ;  $Q \leftarrow Q \cup \{i\}$ ;  $\text{over}[i] \leftarrow 0$   $\triangleright n_i$  fixes  $i$ 's constant part
12:  end for
13:  while a server is free and  $Q \neq \emptyset$  do  $\triangleright$  dispatch epoch
14:     $E \leftarrow \{q \in Q : \text{over}[q] \geq \min(B_0 + \gamma n_q + \eta k(t - a_q), B_{\max})\}$ 
15:    if  $E \neq \emptyset$  then
16:       $j \leftarrow$  the minimum-rank member of  $E$   $\triangleright$  guard fires
17:    else
18:       $j \leftarrow A(Q, t)$   $\triangleright$  base policy decides
19:    end if
20:    dispatch  $j$  on the free server;  $W[j] \leftarrow t - a_j$ ;  $Q \leftarrow Q \setminus \{j\}$ 
21:  end while
22: end while

```

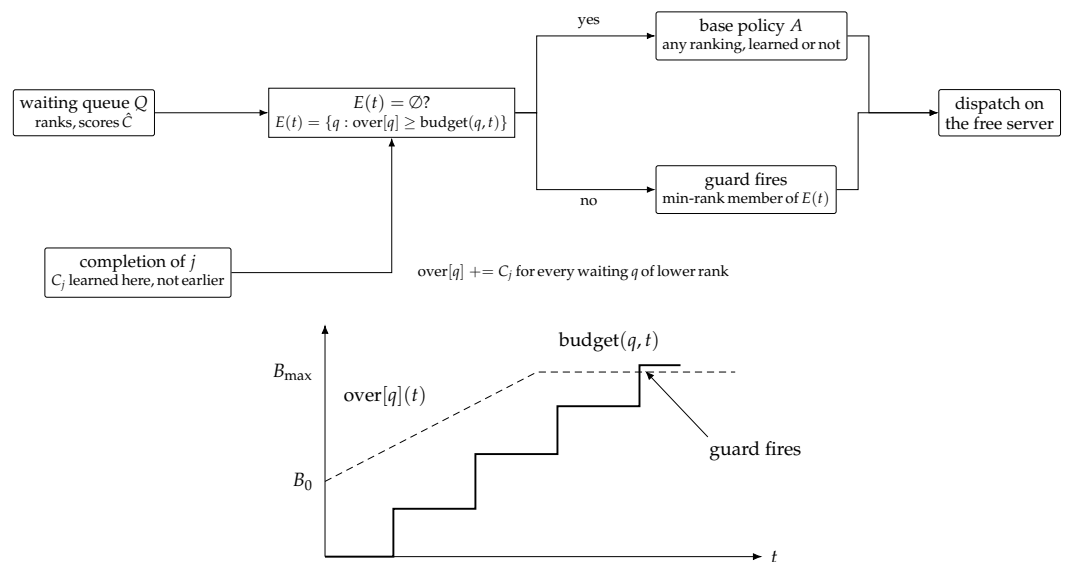


Figure 2. The guard of Algorithm 1 wrapped around a base policy (above) and the accounting it runs for one waiting job (below). Only completed work is charged, because C_j is unknown until j finishes, so $\text{over}[q]$ is a step function with a step at each completion of a higher-ranked job; the first dispatch epoch at which the step function reaches the budget sends q next. The budget drawn is the age-relative shape, rising at rate ηk from B_0 and capped at $B_{\max}$; the guarantee depends on the cap alone, so any curve under the dashed ceiling carries the same promise. The lower panel is schematic. Sources: the definitions in Section 5.2; $B_{\max} = k(G - (3 - 2/k)L)$ is Corollary 3.

$(3 - 2/k)L$; below that threshold no positive budget is admissible, which is not an artefact of the wrapper but a floor of order L that Section 6 shows to be unavoidable for any policy learning service times at completion. The promise G follows from $B_{\max}$ alone. The shape parameters B_0 , γ and η decide how much of the base policy's benefit survives and which jobs pay for it, so we choose them on a validation trace that excludes the test period. The rule is fixed before the grids are run: three pre-stated grids, one per shape, 258 points in all; a point is feasible when its harm is at most $G/2$ in every one of the 15 validation cells; the objective is the deadline-window gap closed in the worst cell, with ties going to smaller worst harm and then to smaller B_0 , η , γ . $\text{Guard}(G)$ is the winner over the three grids together; the winner inside each grid is reported as an ablation row. Table 6 gives the grids and what they selected.

Cost per event.

Algorithm 1 is written for readability rather than for execution, since its inner loop over waiting jobs at each completion is linear. We keep $\text{over}[q] = TC - F(q)$ instead, where TC is the total completed work and $F(q)$ the prefix sum over ranks up to q of completed work, maintained in a Fenwick tree, so that a completion is one point update and a query one prefix sum. The firing test is a segment tree over the window of waiting ranks supporting suffix range-add and the query "leftmost leaf with value at least T ", after rewriting $\text{over}[q](t) \geq B_{0,q} + \eta k(t - a_q)$, where $B_{0,q} = B_0 + \gamma n_q$ is fixed at q 's arrival, with $\eta k = e_n/e_d$ in lowest terms as $e_n a_q - e_d B_{0,q} - e_d F(q) \geq e_n t - e_d TC$, whose left side is a per-job value changed only by range-add at completions and whose right side depends on t alone. The capped channel needs only the head, since at a fixed threshold over is non-increasing in rank. All budget arithmetic is carried in integer microseconds, so the test is exact and no floating-point clock enters the schedule. Each event costs $O(\log n)$.

What is charged, and why no service time is needed at dispatch.

The budget is charged the true executed work of overtakers, and only when they complete. At the instant the guard decides, it has read C_j only for jobs that have already finished, and a job still in service contributes nothing. This is what makes the wrapper implementable in a system that learns a job's cost at its completion, and it is also the reason the bound carries an additive term rather than an exact accounting. When the last overtaker of i is dispatched, up to $k - 1$ further overtakers may be in service with their work not yet charged, and the overtaker being dispatched is not charged either, so the work that actually passes i can exceed the budget by at most kL . Under the timeout at L , executed work means $\min(C_j, L)$, which is what the log records and what the guard charges, so no assumption on raw sizes enters.

Why the budget is measured in work and not in positions.

Earlier bounded-overtaking mechanisms limit how many times a job may be passed, which is the currency of the finite-skip and Nudge- K rules [26,27,29]. Ten overtakers that each run for the full limit L and ten that run for ten milliseconds carry the same count and delays that differ by three orders of magnitude, so a count limit bounds nothing unless a size bound is imposed as well, and with sizes bounded by L it is loose by the ratio $L/\bar{C}$, one to two orders of magnitude on our traces. Remark 2 proves both directions and the one formula the count rule does yield, which Section 8 runs as a baseline; the identity of Section 6 is what makes the choice forced rather than preferred, since the right-hand side it puts the excess equal to is measured in work.

5.3. Compared Policies

Table 3 names the policies evaluated in Section 8 and fixes the names used in every table of this paper. Guard(G) wrapped around SPJF-E is the method; the others separate its parts and attack it.

Table 3. Policies compared in the experiments, and the name each carries throughout.

Name	Role
FCFS	Deployed baseline, and the reference against which the guarantee is stated
SJF	Sorts waiting jobs by their true service times. Undeployable reference policy, not an optimum; the denominator of the fraction of the gap closed
SPJF-E	Ranks by the expected-cost score of Section 5.1, unguarded. Transmission from prediction quality to waiting time
SPJF-log	The same pipeline fitted on the log scale, and the other candidate scores of Section 5.1, run through the same simulator
Guard(G)	SPJF-E wrapped in the overtake-budget guard at the promise G , with the budget shape the selection rule of Table 6 picks over all three grids together. This is the method
Guard-fixed(G), Guard-age(G), Guard-queue(G)	The same wrapper and the same promise, with the shape the rule picks inside the constant, age-relative and queue-length grid respectively. Ablation rows: they isolate what the shape is worth at a fixed cap, and one of them coincides with Guard(G) at each promise
Fixed(G)	The same wrapper at the equal-promise budget $\text{budget} \equiv B_{\max}$. It carries the promise G but not the harm constraint, and the selection rule rejects it as infeasible; it is reported as the comparator that spends the whole cap
Skip(G)	Position-count guard, finite-skip family, counted at dispatch with $N = Gk/L - (2k - 2)$ from (2), the largest count that still promises G under that accounting
Corrupted scores	SPJF-E and Guard(G) under a reversed order, a random order, and a score that calls the truly longest 1% the shortest; tests the guard where the base policy is worst
Counterexamples	Retained traces from two audits and adversarial review; every job must satisfy the bounds of Section 6

Every simulation that includes the guard asserts the per-job bound of Section 6 for each job separately, inside the run rather than afterwards on summary statistics.

6. Theory

Everything below is pathwise: one arrival sequence, one vector of service times, one realisation of whatever randomness the base policy uses. No stationarity, no independence and no assumption about score quality enters any proof, and the upper bounds hold against a base policy that knows the future. An identity ties a job’s delay relative to first-come first-served to the work that overtook it; two bounds turn that identity into the guarantee of Algorithm 1; and a set of constructions shows that each constant in those bounds is an exact supremum. The two main proofs are given here with their key steps; the tightness constructions and the shorter proofs are in Appendix A.

6.1. Conventions

We use the model of Section 3 throughout, and fix three readings the proofs depend on. The dispatches at one instant form a *phase*, ordered as the policy performs them, and the dispatch sequence behind $j \prec i$ is the concatenation of the phases; first-come first-served dispatches in increasing rank inside a phase. A job dispatched in the same phase as i executes no work during $[a_i, s_i^P)$, that interval ending at the instant of the phase, yet

contributes its whole C_j to In_i . Remaining work at a_i is read *after* the arrivals at a_i , so a job of rank below i arriving at exactly a_i counts with its full service time.

6.2. The workload gap between two policies

Write $U_Q(t)$ for the unfinished work at time t under policy Q , counting every job with $a_j \leq t$, and $N_Q(t)$ for the number of jobs present. The extremal properties of first-come first-served for the unfinished-work vector are classical [37,38]; what carries the single-server argument to $k > 1$ is a two-sided bound between two arbitrary policies rather than an ordering.

Lemma 1 (Workload comparison). *Let A and B be non-preemptive work-conserving k -server policies run on the same input. Then $|U_A(t) - U_B(t)| \leq (k-1)L$ for every t , and the inequality is strict: $|U_A(t) - U_B(t)| < (k-1)L$. The constant is the exact supremum, approached but never attained.*

The difference $D = U_A - U_B$ is continuous and piecewise linear, and it cannot increase at any instant at which U_A exceeds $(k-1)L$, since more than $k-1$ jobs are then present under A and every server of A is busy; tracking D back to the last instant at which U_A was below the threshold gives the bound. The proof, the strictness, and the cascade that shows the constant cannot be lowered are in Appendices A.1 and A.2.

6.3. The net-overtake identity

For a job i and a policy Q , let R_i^Q be the remaining work at time a_i , under Q , of the jobs of rank below i , read after the arrivals at a_i ; and let ρ_i^Q be the remaining work at s_i^Q of the jobs dispatched strictly before i in Q 's dispatch sequence that are still in service then. Write

$$\Delta_i = k(W_P[i] - W_{\text{FCFS}}[i]) - (\text{In}_i - \text{Out}_i).$$

Theorem 1 (Net-overtake identity). *For every non-preemptive work-conserving k -server policy P , every input and every job i ,*

$$\Delta_i = (R_i^P - R_i^{\text{FCFS}}) - \rho_i^P + \rho_i^{\text{FCFS}}, \quad |\Delta_i| \leq 2(k-1)L,$$

equivalently $|W_P[i] - W_{\text{FCFS}}[i] - (\text{In}_i - \text{Out}_i)/k| \leq (2 - 2/k)L$. At $k = 1$ the error term vanishes and $W_P[i] - W_{\text{FCFS}}[i] = \text{In}_i - \text{Out}_i$ exactly.

Proof. *Busy-period accounting.* Job i waits throughout $[a_i, s_i^P)$, so by work conservation every server is busy on that interval and exactly $k W_P[i]$ units of work are executed there, none of it on i . (If $s_i^P = a_i$ the interval is empty and both sides vanish.) Split that work by rank.

A job j of rank above i has $a_j \geq a_i$, so it executes inside the window only if it is dispatched inside it, that is only if $j \prec i$; conversely every such j is dispatched at or before s_i^P . The work executed on jobs of rank above i is therefore $\text{In}_i - \rho^{\text{new}}$, where ρ^{new} is the remaining work at s_i^P of those overtakers not finished by then; a same-phase overtaker contributes C_j to both terms and 0 to the executed work, as it must.

Every job of rank below i is present at a_i , with total remaining R_i^P . At s_i^P their total remaining is Out_i , the jobs not dispatched before i each carrying their full service time, plus ρ^{old} , the unfinished part of those dispatched strictly before i , so the work executed on them is $R_i^P - \text{Out}_i - \rho^{\text{old}}$. Adding the two parts,

$$k W_P[i] = R_i^P + \text{In}_i - \text{Out}_i - \rho_i^P, \quad \rho_i^P := \rho^{\text{old}} + \rho^{\text{new}}. \quad (1)$$

Bounding ρ . The jobs counted in ρ_i^P occupy distinct servers, and none of them occupies the server that takes i , so there are at most $k - 1$ of them and each has remaining work at most L : $0 \leq \rho_i^P \leq (k - 1)L$.

The reference policy. Under first-come first-served, $\text{In}_i = 0$, since a job of rank above i dispatched before i would have to be chosen at an instant at which i waits, and $\text{Out}_i = 0$, since at s_i^{FCFS} every job of rank below i has already been dispatched. So (1) reads $k W_{\text{FCFS}}[i] = R_i^{\text{FCFS}} - \rho_i^{\text{FCFS}}$ with $0 \leq \rho_i^{\text{FCFS}} \leq (k - 1)L$.

Comparison. At a_i , after the arrivals there are processed, the two systems contain the same jobs, and those of rank i or above are untouched in both, so $U_P(a_i) - U_{\text{FCFS}}(a_i) = R_i^P - R_i^{\text{FCFS}}$ and Lemma 1 gives $|R_i^P - R_i^{\text{FCFS}}| < (k - 1)L$. Subtracting the two instances of (1) gives the identity. Each direction uses only two of the three ranges: upward $R_i^P - R_i^{\text{FCFS}} \leq (k - 1)L$ and $\rho_i^{\text{FCFS}} \leq (k - 1)L$ with $\rho_i^P \geq 0$, and downward the mirror image, so the slack is $2(k - 1)L$ and not $3(k - 1)L$. At $k = 1$ all three terms vanish. $\square$

Corollary 1 (A work budget is sufficient). *If a policy keeps $\text{In}_i - \text{Out}_i \leq Z$ for every job, in particular if it keeps $\text{In}_i \leq Z$, then $\text{excess}_P[i] \leq Z/k + (2 - 2/k)L$ for every job.*

6.4. The constant $2(k - 1)L$ is exact

The error term of Theorem 1 looks like an artefact of the proof, and it is not, although neither search nor real input shows it. Exhaustive and random search over millions of small instances never exceeds $(k - 1)L$ by more than one time unit, and on the 17.6-million-job trace of Section 8 the largest residual reaches 0.485 to 0.521 of $2(k - 1)L$, again about $(k - 1)L$ (Table 11). The extremal instances are geometric cascades needing hundreds of jobs.

Theorem 2 (Tightness of Theorem 1). *For every $k \geq 2$ and every $\varepsilon > 0$ there are inputs, work-conserving policies P and jobs i with $\Delta_i > 2(k - 1)L - \varepsilon$, and there are others with $\Delta_i < -2(k - 1)L + \varepsilon$. Hence $\sup |\Delta_i| = 2(k - 1)L$, a supremum that is never attained.*

Both witnesses start from the cascade of Appendix A.2, which drives $U_P - U_{\text{FCFS}}$ to $(k - 1)L(1 - ((k - 1)/k)^m)$ in m rounds. The upward witness then injects $k - 1$ long jobs, the victim, and a stream of unit jobs the base policy prefers, so that first-come first-served starts the victim at once with $\rho_i^{\text{FCFS}} = (k - 1)L$ while P holds it until the stream runs out; the downward witness is the mirror image, with P dispatching $k - 1$ jobs of service L in the victim's own phase. Appendix A.3 gives both in full, with the exact values they attain. Non-attainment follows from Lemma 1.

Remark 1 (The two directions are not alike). *Upward, the extremal value is a statement about delay: on the witnesses we have computed exactly, none of In_i is same-phase work and the victim's own excess is about $+3L$. Downward, half of the extremal value is an accounting convention. There $\text{In}_i = (k - 1)L$ consists entirely of overtakers dispatched in the victim's own phase, which execute nothing while it waits; the same-phase share of $|\Delta_i|$ is 50% to 63% at $k = 2, 3, 4$ (Appendix A.3), and the downward victim's excess is negative, since it starts earlier under P than under first-come first-served. If In_i is redefined as the work of higher-ranked jobs actually executed during $[a_i, s_i^P)$, Theorem 1 holds with the same constant and the same proof, the upward supremum is still $2(k - 1)L$, and the best downward witness we have falls to $(k - 1)L$. We keep the dispatch-sequence convention because In_i is the quantity a scheduler can meter: the guard charges completed work, which lower-bounds In_i but not its executed-work variant.*

6.5. Necessity, and why positions are the wrong currency

Theorem 1 also runs backwards. A policy that promises anything about individual jobs is forced to bound overtaken work.

Lemma 2. *Let i be a job, and suppose $\text{excess}_P[i] \leq G$ and $\text{excess}_P[j] \leq G$ for every job j counted in Out_i . Then $\text{Out}_i \leq k(G - \text{excess}_P[i] + L)$.*

Proof. Every job j counted in Out_i has rank below i , so first-come first-served dispatches it before i and $s_j^P \leq s_j^{\text{FCFS}} + G \leq s_i^{\text{FCFS}} + G$; it is also dispatched at or after s_i^P . All of them therefore start inside a window of length $\delta = G - \text{excess}_P[i] \geq 0$. Of the jobs one server starts inside such a window, all but the last complete inside it, so their total service is at most δ and the last adds at most L . $\square$

Corollary 2 (A work budget is necessary). *If P satisfies $\text{excess}_P[j] \leq G$ for every job on some input, then $\text{In}_i \leq kG + (3k - 2)L$ for every job i .*

Proof. Theorem 1 gives $\text{In}_i - \text{Out}_i \leq k \text{excess}_P[i] + 2(k - 1)L$; add Lemma 2, whose hypothesis holds because the guarantee is assumed for every job, and the terms in $\text{excess}_P[i]$ cancel. $\square$

Together the two corollaries say that bounding In_i and offering a per-job guarantee are the same requirement, up to an additive $O(kL)$ in the budget. Neither says that Algorithm 1 is the least restrictive rule with its guarantee, and we do not claim it.

Remark 2 (Counting overtakes is neither sufficient nor necessary). *Bounded-overtaking mechanisms in the literature limit how many times a job may be passed [26,27,29]. Without a bound on service times one overtake makes the excess arbitrarily large: at $t = 0$ let $k - 1$ jobs of service X , then a job of service ϵ , then one more job of service X arrive together; first-come first-served starts the victim immediately, a policy that takes the last job instead fills every server, and the victim waits X with an overtake count of 1. Conversely a count bound is not necessary: with $k = 1$, $L = 2M$, a victim of service L arriving first and M jobs of service 1 arriving with it, running the short jobs first gives every job an excess of at most $M = L/2$, using $\text{In}_i = kG$ exactly at the guarantee $G = L/2$, while the overtake count M exceeds any fixed limit. With services bounded by L a count wrapper does give a guarantee, and how the count is charged decides which one. Charge it when an overtaker is dispatched, the reading a position counter in the prior art implements. Every overtaker of i is dispatched at an instant at which no job fires, since a firing dispatch serves the queue head and the head is not an overtaker of i ; so at most N overtakers are dispatched while i waits, $\text{In}_i \leq NL$, and Corollary 1 gives*

$$W_P[i] \leq W_{\text{FCFS}}[i] + (N + 2k - 2)L/k, \quad (2)$$

which is the accounting and the formula used wherever this baseline appears below. Inverting it, a promise G admits $N = Gk/L - (2k - 2)$ passes. Charging the count at completion instead leaves up to $k - 1$ overtakers in service and the one being dispatched uncounted, which costs a further L on the right and gives the baseline fewer passes at the same promise. Either accounting charges every overtaker the worst case L , so the rule is loose against a work budget by the ratio $L/\bar{C}$, one to two orders of magnitude on our traces.

6.6. The guarantee of the guard

We now bound what Algorithm 1 does. Nothing below restricts the base policy, which may be randomised, learned, or clairvoyant.

Theorem 3 (Guard bound). *Fix any base policy A and any cap $B_{\max} \geq 0$, and run Algorithm 1 with any budget rule that respects the cap, that is, with any function $\text{budget}(q, t)$ satisfying $0 \leq \text{budget}(q, t) \leq B_{\max}$ for every waiting job q and every dispatch epoch t ; the rule may depend on the job, may decrease over time, and may take different values at two dispatches made at the same instant. For every input and every job i , writing $W = W_P[i]$ for the wait under the wrapper,*

$$\text{In}_i < B_{\max} + kL, \quad (3)$$

$$W < W_{\text{FCFS}}[i] + B_{\max}/k + (3 - 2/k)L, \quad (4)$$

where (3) is asserted whenever i has at least one overtaker.

Proof. For (3), let j be the overtaker of i dispatched last in the dispatch sequence and let t be its instant. Because dispatches are sequential and i is dispatched after j , job i is waiting at the moment j is dispatched; this includes the case $t = s_i^P$, in which j is dispatched in i 's own phase, just before i . Since j is dispatched, either $E(t)$ is empty or its minimum-rank member is j ; in both cases $i \notin E(t)$, because i has the smaller rank. Hence $\text{over}[i](t) < \text{budget}(i, t) \leq B_{\max}$.

The overtakers of i dispatched strictly before j split into those completed by t , whose total service is $\text{over}[i](t)$, and those still in service at t ; the latter occupy distinct servers, none of them the server taking j , so there are at most $k - 1$ of them, each of service at most L . Adding j itself gives $\text{In}_i < \text{budget}(i, t) + kL \leq B_{\max} + kL$.

For (4), Theorem 1 with $\text{Out}_i \geq 0$ gives $k(W - W_{\text{FCFS}}[i]) \leq \text{In}_i + 2(k - 1)L$; substituting (3) and dividing by k gives the claim, strictly. If i has no overtaker then $\text{In}_i = 0$, the excess is at most $(2 - 2/k)L$, and the bound holds strictly because $L > 0$ in the model of Section 3. $\square$

Nothing in the proof reads the budget rule except through the cap, so the shape of the budget below the cap is a free choice and the promise is unaffected by it. One shape carries a second bound, in which the allowance grows with the wait the job has already had rather than being flat in it.

Theorem 4 (Multiplicative bound for the age-relative shape). *Suppose in addition that $\text{budget}(q, t) = \min(B_0 + \eta k(t - a_q), B_{\max})$ with one constant $B_0 \geq 0$ for every job and $\eta \in [0, 1)$. Then for every input and every job i ,*

$$\text{In}_i < B_0 + \eta kW + kL, \quad (5)$$

$$(1 - \eta)W \leq W_{\text{FCFS}}[i] + B_0/k + (3 - 2/k)L, \quad (6)$$

with (5) asserted whenever i has at least one overtaker.

Proof. The argument for (3) gives $\text{In}_i < \text{budget}(i, t) + kL$ at the instant t of i 's last overtaker, and $t \leq s_i^P = a_i + W$ gives $\text{budget}(i, t) \leq B_0 + \eta kW$, which is (5). Theorem 1 with $\text{Out}_i \geq 0$ then yields $kW - \eta kW \leq kW_{\text{FCFS}}[i] + B_0 + (3k - 2)L$; divide by k . $\square$

Remark 3 (Why the multiplicative bound is claimed for one shape only). *The same computation goes through when B_0 is replaced by a quantity $B_{0,i}$ fixed at i 's own arrival, and then bounds W in terms of $B_{0,i}$. The queue-length shape of Section 5 sets $B_{0,q} = B_0 + \gamma n_q$ with n_q the number of jobs waiting when q arrives, so $B_{0,i}$ varies from job to job. Stated with the common B_0 the multiplicative bound is then false, by a margin that grows with γ : take $k = 1$, $L = 1$, $B_0 = 0$ and $B_{\max} = \gamma$; a job that arrives behind a single waiting job has budget γ and can be made to wait exactly γ longer than under first-come first-served, while the right-hand side of (6) is $2/(1 - \eta)$*

whatever γ is. The same happens for any per-job constant. For $\gamma > 0$ we therefore claim Theorem 3 and not Theorem 4. The cap, and with it the promise, is unaffected either way.

A scheduler that learns C_j at completion cannot charge work in progress, so only completed work is charged; the kL in (3) is the price of that and is what forces the additive term. Inverting (4) gives the parameter rule of Section 5:

Corollary 3. $B_{\max} = k(G - (3 - 2/k)L)$ makes Algorithm 1 offer the service guarantee G , whatever the budget shape below the cap and whatever the base policy A . The choice is available exactly when $G > (3 - 2/k)L$.

6.7. Both constants of the guarantee are exact

Theorem 5 (Tightness of Theorem 3). Take the constant budget $\text{budget}(q, t) \equiv B = B_{\max}$, and write $c(k) = \sup(k \text{ excess}_P[i] - B)/L$ over all inputs, base policies and budgets; by Theorem 3 the supremum over all budget rules that respect the cap B is the same number. Then

1. the term B/k is attained: on an explicit family the excess is exactly $\lceil B/k \rceil$, hence B/k whenever k divides B ;
2. $c(k) = 3k - 2$ exactly, for every $k \geq 1$, as a supremum that is never attained. Equivalently, the additive constant $(3 - 2/k)L$ of (4) cannot be lowered for any k .

Part (i) is a queue of k jobs of service L , the victim, and a stream of unit jobs: the guard fires at $L + \lceil B/k \rceil$. For part (ii) the upper bound is Theorem 3, strictly; the lower bound runs the cascade for m rounds and then injects a sequence timed so that $\text{over}[i]$ stops one unit short of the budget twice in succession, letting the base policy fill every server a third time before the guard fires. Appendix A.4 gives both constructions and the values they reach.

Both sources of slack in Theorem 3, the constant of Theorem 1 and the kL of (3), are saturated on the same instance, which is what makes the composition tight rather than each part separately, and the excess is real waiting time rather than bookkeeping: every overtaker completes before the victim starts, so the same-phase share of In_i is zero and the executed-work convention of Remark 1 gives the same number. The additive constant $(3 - 2/k)L$ grows with k , from L at $k = 1$ towards $3L$, and search does not see this: a hill-climb over instances of at most 20 jobs suggests a flat value near $1.75L$, while the witness above needs 396 jobs at $k = 2$.

6.8. When the guard is invisible

Proposition 1 (Consistency, one direction). Run the base policy A alone. If $\text{over}[q](t) < \text{budget}(q, t)$ for every waiting q at every dispatch epoch of that run, then the wrapped run is identical to it, with the same dispatch sequence and the same waits. Consequently, for a constant budget B , each of the following implies $\text{guard}(A, B) = A$: $\max_i \text{In}_i^A < B$; or A itself satisfies $\text{excess}_A[i] \leq G$ on this input and $B > kG + (3k - 2)L$.

The proof inducts on dispatch epochs and is in Appendix A.5, with the two counterexamples that show the converse fails and that there is no multiplicative consistency: the guard can fire and happen to choose what the base would have chosen, and the ratio of mean waits between the wrapper at $B = 0$ and a base ordering by service time is unbounded in L .

6.9. Which assumptions can be dropped

Proposition 2 (Relaxations). In the setting of Theorem 1 and Theorem 3:

1. No global bound on service times is needed, provided the prefix maximum $\Lambda(t) = \max\{C_j : a_j \leq t\}$ is placed correctly. The upward direction of Theorem 1, and hence Corollary 1 and (4), hold with L replaced by $\Lambda(a_i)$; the downward direction needs $\Lambda(\max(s_i^P, s_i^{\text{FCFS}}))$ and Corollary 2 needs $\Lambda(s_i^{\text{FCFS}} + G)$, both being false with $\Lambda(a_i)$.
2. Servers may have different speeds. With speeds $v_1, \dots, v_k$ and $V = \sum_m v_m$, Theorem 1 holds with k replaced by V on the left and the same $2(k-1)L$ on the right, and the guard satisfies $W_P[i] \leq W_{\text{FCFS}}[i] + B/V + (3k-2)L/V$: the budget is divided by total capacity, not by the server count.
3. The timeout is already in the model. Section 3 defines C_j as executed work, $\min(C_j, L)$ for a job stopped at the limit, so every currency above $(\text{In}_i, \text{Out}_i, \text{over}[q](t))$ and L is in executed work and no assumption on raw service demands is needed. Read in raw work the statements are false, a stopped job contributing more to In_i than is ever executed.
4. Setup times survive. If a setup of length σ is paid before every service under both policies, absorb it into the job as $C_j' = C_j + \sigma$; the model is unchanged and every result holds with $L \mapsto L + \sigma$.
5. Work conservation cannot be dropped. If servers may idle while jobs wait, let Γ_i^P be the server-idle time accumulated during $[a_i, s_i^P)$ while at least one job waited. Then (1) becomes $kW_P[i] = R_i^P + \text{In}_i - \text{Out}_i - \rho_i^P + \Gamma_i^P$, and nothing survives without a bound on Γ : with $k = 1$, pausing the server for T while i waits leaves $\text{In}_i = \text{Out}_i = 0$ and makes the excess T . Release constraints are the same phenomenon: if rank is keyed on arrival while eligibility starts later, the identity fails by exactly the forced idle time, and ranking on release repairs it.

6.10. A floor of order L , and what a guarantee costs

Proposition 3 (Impossibility). Let P be any non-preemptive work-conserving policy that observes a job's service time only at its completion. Then there are inputs on which some job has $\text{excess}_P[i] \geq L$. In particular no wrapper with a positive budget can promise $G < L$, and at $k = 1$ the bound $B + L$ of Theorem 3 is the exact supremum of the excess, approached and not attained.

Proof. Into an empty system, at $t = 0$, let the victim i of rank 0 and k further jobs arrive together, all of service L . First-come first-served starts i at once. Work conservation permits P to fill all k servers with the higher-ranked jobs, and P cannot be dissuaded from doing so, having observed no service time yet, so i waits until the first completion and $\text{excess}_P[i] = L$; any wrapper has $\text{over}[i](0) = 0$ and does not fire at $t = 0$. At $k = 1$, (3) gives excess $< B + L$ strictly, and a family approaching it is in Appendix A.6. $\square$

The guarantee also caps how much reordering can buy at all, and not only how much it may cost a single job.

Theorem 6 (Price of the guarantee). Take m jobs of service L and n jobs of service $s < L$, the long ones ranked first, all arriving at $t = 0$, and let P be any non-preemptive work-conserving policy with $\text{excess}_P[j] \leq G$ for every job. Then

$$\frac{\overline{W}_{\text{FCFS}} - \overline{W}_P}{\overline{W}_{\text{FCFS}} - \overline{W}_{\text{SJF}}} \leq \frac{kG + (3k-2)L}{ns} + \varepsilon,$$

with $\varepsilon = O((m+n)/(mn)) \rightarrow 0$ as m, n grow.

The proof is in Appendix A.7. The right-hand side is capacity times guarantee, divided by the work that wants to overtake; it is linear in G and saturates at 1 when kG reaches the total service of the impatient class. Because it applies to every policy with the guarantee and not only to Algorithm 1, it describes the trade-off an operator faces

rather than a weakness of this wrapper. At $k = 1$ the achievable fraction is exactly $\min(1, G/(ns))$.

Every constant above is a multiple of L , and Proposition 3 says no mechanism of this kind can do better, so the guarantee is informative when L is small relative to the waits an operator cares about and empty when it is not. Section 8.7 makes that boundary quantitative on six workloads, one of which falls outside it.

7. Traces and Workload Characterisation

7.1. The Traces

The validation scenario is an educational one: two programming-assignment judging platforms supply the demand traces, and four further workloads bound the claim from outside that setting. What each trace can be used for is decided by whether its arrival times are real, whether the queue wait is recorded, and how long a job may run there (Table 4).

Table 4. The traces, their role and what they record. “Arrivals” says whether the arrival instants are recorded rather than drawn; “Waits” whether the queue wait of each job is itself recorded.

Trace	Role	Size	Time fields	Arrivals	Waits
CodeBench [13]	main demand trace	4,611,325 blocks, 863,149 graded submissions in the development pool	one stamp per block, the end of the run	yes (as end – C)	no
ACcoding [14]	second judging platform	4,046,652 rows, 3,237,322 non-sealed	none; submission id only	no, drawn	no
Serverless invocations [7]	method applies outside education	1,980,951 invocations over 14.000 d	end stamp and duration	yes	no
CI pool A [57,58]	simulator validation, light tail	191,970 runs over 21 d	created, scheduled, started, resolved	yes	yes
CI pool B	second validation pool	30,702 runs over 6.99 d	as above	yes	yes
Compute farm [15,16]	non-applicable case	9,054,066 jobs over 30.00 d, 8,441,207 serial with positive run time	submit, wait, run	yes	yes
OULAD [59]	calendar-driven load elsewhere	9 course-presentations, daily counts	day	—	—

Both CI pools and the compute farm carry a recorded queue wait. We validate the simulator against the CI pools, whose capacity is a countable set of physical machines that changes only on reimaging or quarantine. The second judging platform has no timestamp column, only a submission id, so job order inside a group and job sizes are real while every arrival instant is drawn. The correlation between a group’s first id and its recorded start time is 0.9995 over 469 groups, but in 207 of 468 consecutive transitions the id ranges overlap, so the id is an ordering and not a clock. Its results are a check that the ordering mechanism transfers, not a wait in seconds.

7.2. What the Fields of the Main Trace Mean

Several fields of that trace needed a reading before they could carry a queueing model.

The timestamp on an execution block is the moment the result was written, not the moment the code was received. Among later-period runs the gap to the same user’s previous event grows one for one with the cost (slope 0.99 of the median gap on C) while the gap to the next event barely moves (slope 0.16), and the feedback after a user kills a run arrives within about 0.1 s whatever its length. A result stamp predicts both readings; a receipt stamp neither. We take $done_j$ to be the recorded stamp and reconstruct the arrival

as $a_i = t_i - C_i$, carrying the opposite reading through the pipeline as a sensitivity with its own features and models.

Stamps are recorded to the whole second while costs are recorded to the microsecond, which opens a leak: the fractional part of a job's cost appears in the gap between its own stamp and the next event, so a feature like "seconds since the user's previous record" reads the label off the clock. We close it by replacing t with $t + u$, where $u \in [0, 1)$ is drawn deterministically from a fixed key and the record's identifier. A classifier given only the fractional part of that gap separates heavy from light jobs with AUROC 0.8091 on the whole-second clock and 0.4499 after the jitter, while the forward models of the two clocks stay within 0.008 of AUROC of each other, so closing the channel costs almost nothing.

The cost field is wall-clock occupancy of one evaluation including interpreter start-up, read from the 10th percentile of correct submissions (0.12–0.18 s across the two periods) and from its weak dependence on the number of test cases (median $0.196 + 0.007n$ s in the later period). The platform limit is 60 s: the last run to reach it ends at 2019-08-28 10:32, after which the largest observed cost is 59.97 s, with 13 runs in $[59, 60)$ s and none above. Before that date there was no fixed limit, the longest run is 8,901 s, and the 1,189 runs that end with a kill are almost all above 360.8 s, pointing at a container watchdog rather than judging demand. Those periods stay out of the primary pool, and where used at all they carry the later limit, $C^{\text{cap}} = \min(C, 60)$ (Table 5).

Groups of runs by the same user that end in the same second are 7.53% of blocks and 26.28% of capped work; they overlapped and stretched each other, so we neither delete them silently nor treat each as an independent arrival. Every prediction table is repeated on rows outside such groups, and the scheduling trace is rebuilt twice, once with each group collapsed into one job and once with the groups removed (Table 14). A further 1,671 blocks record a cost of exactly zero with empty outputs; they leave the simulated trace and stay in the features, the labels and every total. The 2019-2 archive stops at 2019-09-19, months before that term's assessments end, and 2020-1 is short by about 3% of its blocks; both serve per-submission modelling and are excluded from anything that treats a term as a whole. Against the published release statistics we match the class count in 18 of 18 terms and the student and exercise counts in 16 of 18, and parse 4.08% fewer execution blocks than the official column sums, whose printed totals disagree with those same columns by -248 exercises and $+246,246$ blocks. We quote the columns.

7.3. The Original Platform Has No Shared Queue

The platform that produced the main trace gives each logged-in student a dedicated container and runs their code inside it, so submissions there did not queue behind one another. We use the log as a demand trace: arrival instants and per-job costs are real; the server pool, the queue and the order are our design. Nothing in this paper claims that the original platform slowed down before a deadline.

Three things support the design. The first is that a fixed pool of judging hosts on one queue is a standard deployment. A widely used contest judging system recommends "one judgehost per 20 teams" [2] on dedicated machines, and the autograder study of Section 1 runs a configurable number of concurrent graders, replays the busiest hours before a shared deadline and measures the queue wait [3]. Dedicated per-entity capacity is also known to sit idle in the workloads this one resembles. In the serverless characterisation behind our invocation trace, 45% of applications are invoked once an hour or less and 81% once a minute or less, so the authors report that the cost of keeping them warm relative to their billable execution time can be prohibitively high [60]; and a cluster manager that measured the alternative directly reports that splitting each large user's workload into

its own cell would need 2 to 16 times as many cells and 20 to 150% more machines [61]. Neither source is about judging, and neither licenses a number of our own.

The second is a measurement on the same demand. Replaying the overlay of Section 7.4 under the dedicated design, the 99th percentile of containers alive at once is 1,636 and each container executes code for 0.12% of the time it is alive; since only 11% to 35% of logins have a logout written after them, session boundaries are reconstructed, and across six reconstructions that percentile lies between 1,003 and 2,525 containers and the utilisation between 0.07% and 0.17%. A shared pool holds the 99th percentile of deadline-window waits under 1 s with 14 servers under FCFS and 13 under the predictor-ordered policy. Pooling is worth two orders of magnitude here and the scheduler is worth one server (14 to 13 at 1 s, 11 to 10 at 5 s, 8 to 7 at 30 s). It does not support a claim that the platform ever queued, the pooled side being simulated, nor that the containers are over-provisioned, since they also run the editing session the pool does not replace, nor that any of it is a cost saving.

The third is that the simulator reproduces waits somebody else recorded. The CI logs carry the queue wait of every run, so we replay the recorded arrivals and services through our own k -server non-preemptive kernel. Two quantities are fitted: the effective capacity k_{eff} , the shortfall against the listed machine count being machines that are up but not claiming work, and a setup term, the median short gap between consecutive runs on the same machine, added to every service because a machine is not reusable the instant a task resolves. Fitting both on the window and then reporting agreement on that same window would be circular, so we refit both on one chronological part of each window and test on the other, five splits over the two pools.

The capacity is stable across splits. On pool A the three splits choose 155, 155 and 157 of 173 listed machines, and on pool B the two splits choose 96 and 92 of 105; the setup term moves between 54.2 and 55.3 s on the first and between 126.2 and 128.6 s on the second. The fit is sharp rather than flat: on a step-of-one grid only 5 to 8 values of k put the simulated mean wait within 10% of the recorded one. With those parameters frozen, the held-out part comes back at 0.90 to 1.14 times the recorded mean wait, 0.86 to 1.16 at the 99th percentile and 0.89 to 1.28 at the 99.9th, with a per-hour mean-wait correlation of 0.936 to 0.981, a per-job rank correlation of 0.940 to 0.963, and a median per-job absolute error of 16 to 84 s.

One quantile does not survive. The 90th percentile lands at 0.81 to 1.21 of the recorded one, and on the larger pool it is below the recorded value on all three splits while the 99th is above it, so the simulated distribution is too steep in its upper middle; the two-term objective hides this in sample by trading the mean against the p90, which is why we fit out of sample and say so. What the fit absorbs is nonetheless a property of the pool rather than an artefact of fitting: told instead that every listed machine is available, the same simulator underestimates the held-out mean wait by a factor of 1.3 to 2.6 and its per-hour correlation falls to 0.75. The circularity of the original whole-window fit can be read off directly: on pool A the whole-window and held-out rows are almost identical because every split picks nearly the same capacity, while on pool B the reverse split's held-out capacity gives a mean ratio of 1.14 on its test part where the whole-window capacity gives 0.99.

7.4. Building the Shared Pool

One course does not fill one server, so we overlay. The pool takes only the development terms that ran under the 60 s limit, 40 class-terms in all, and replicates them 44 times, each copy aligned by weekday and hour and shifted by a whole number of weeks drawn per copy. Overlaying whole terms rather than compressing inter-arrival times preserves

the daily and weekly cycle and each job’s position relative to its own deadline. The result is 17,634,760 jobs over 30 weeks and 208.8 days. The three load levels share that trace and differ only in the pool size. Four of the five overlays take $k = 8/5/4$ and the fifth takes $7/5/4$, its work per busy hour being the lowest of the five; on the first overlay that gives busy-hour utilisations of 0.529, 0.847 and 1.059, and 0.502, 0.783 and 0.979 averaged over the five. A deadline window is the 24 hours before an assessment’s own end, and 4,219,556 jobs arrive inside one. Five overlays are week-shifts of the same terms rather than five platforms, and our intervals resample whole weeks, so they cover time and not the choice of terms. A trace built from the last development term alone is not available this way, since it would need 190 copies of its 8 class-terms to load a pool of four.

7.5. What the Load Looks Like

Arrivals follow the course calendar. Predicting the number of executions in an hour 168 hours ahead, a model with lag features alone reaches log-scale RMSE 1.243, one with calendar features alone 1.159, and one with both 1.112; the paired difference between the last two is -0.1476 $[-0.1752, -0.1198]$ over class-week blocks. The corroborating trace shows the same shape elsewhere: clicks in the three days up to an assessment due date are 1.29 to 2.95 times the other test days across its nine course-presentations, and a seven-day-ahead peak warning on distinct active students goes from F1 0.000 under a moving-average baseline to 0.593 once calendar features are added.

Costs are heavy-tailed on every workload except the CI pools, which is where the method’s second applicability condition shows up (Table 5).

Table 5. Share of total executed work carried by the longest jobs.

Trace	Top 1%	Top 5%	Per-job limit
CodeBench, 60 s regime	0.375	0.432	60 s
CodeBench, no-limit periods, raw	0.941	0.960	none
CodeBench, no-limit periods, capped	0.494	0.658	60 s applied
ACcoding, measured cost	0.275	0.628	30.0 s
ACcoding, with the assumed fixed term	0.122	0.290	30.5 s
Serverless invocations	0.443	0.879	286.8 s
Compute farm	0.372	0.628	493,857 s
CI pool A	0.034	0.107	5,066 s
CI pool B	0.052	0.159	4,140 s

The per-job limit in the last column is the unit in which the guarantee of Section 6 is denominated; Section 8.7 compares it with the waits each operator cares about and decides on that basis which of the six workloads the method applies to.

7.6. Sealed Data

Part of every trace was sealed before the method was designed, and is opened once after the method, its parameters and the primary metric are frozen. On the main trace the sealed part is three terms, 2023-1, 2023-2 and 2024-1; on the second judging platform it is the last 20% of the submission-id range, 809,330 rows, of which only the row count and the first id have been read; on the corroborating trace it is the 2014 academic year. No cost distribution, prediction or scheduling result has been computed on any sealed part, and every access to one is entered in a ledger in the repository. Every number in this section and in Sections 4 and 5 is a development number.

8. Experiments

The experiments are organised by the three questions of Section 1. Question 1, whether the cost of a job can be estimated before it runs, is answered in Section 4 and is not repeated here. This section takes the predictors as given and asks what a scheduler

should sort by (Section 8.3), what the per-job guarantee costs and what it buys (Section 8.4), whether the bounds of Section 6 hold on real traces (Section 8.5), and where the method applies (Sections 8.7 and 8.8).

8.1. Setup

All scheduling runs replay the overlaid CodeBench pool built in Section 7.4. The three load levels differ only in the pool size and the five overlays only in the per-copy week shift, and every policy sees the same arrival instants and the same service times, so the comparison is pathwise. Table 6 collects the configuration and the selection protocol; the metrics are defined in Section 3 and the policy names in Table 3. We report the fraction of the FCFS-to-SJF gap closed and the plain percentage reduction against FCFS side by side throughout, because they are different numbers and the first is easy to read as the second.

Table 6. Experimental configuration and the selection protocol for the guard parameters.

Item	Value
Trace and pool	CodeBench development terms under the 60 s limit, overlaid as in Section 7.4; 17,634,760 jobs, 30 weeks, five overlays
Load levels	$k = 8/5/4$ on four of the five overlays and $7/5/4$ on the fifth, whose work per busy hour is the lowest; busy-hour $\rho = 0.529/0.847/1.059$ on overlay 0
Single-server configuration	separate trace, 321 copies, 3,209,347 jobs, busy-hour $\rho = 0.80$ at $k = 1$
Primary metric	99th percentile of the wait among jobs arriving in the 24 h before their own deadline; secondary metrics as defined in Section 3, the gap closed printed beside the percentage reduction
Intervals	paired week-block bootstrap, 2,000 resamples of the 30 whole weeks, one draw of week multiplicities applied to every policy and to all five overlays
Promise to parameter	$B_{\max} = k(G - (3 - 2/k)L)$ with $L = 60$ s, no data used
Budget shapes	one mechanism under one cap, three shapes: constant ($\gamma = \eta = 0$), age-relative ($+\eta k(t - a_q)$), queue-length ($+\gamma n_q$). Three pre-stated grids, $42 + 162 + 54 = 258$ points
Selection rule	a point is feasible if its harm is $\leq G/2$ at each of the three load levels of each of the five validation overlays (15 cells); the objective is the deadline-window gap closed in the worst cell; ties to smaller worst harm, then smaller B_0 , then η , then γ . Guard(G) is the winner over the three grids together; the winner inside each grid is an ablation row
Selected, $G = 300$ s	age-relative: $B_0 = 60k/4$ s, $\eta = 0.50$, $\gamma = 0$; worst-cell gap 0.4251, worst-cell harm 122.8 s against a 150 s limit, 44 of 99 candidate points feasible
Selected, $G = 600$ s	age-relative: $B_0 = 120k/4$ s, $\eta = 0.75$, $\gamma = 0$; worst-cell gap 0.7216, worst-cell harm 293.3 s against 300 s, 70 of 107 feasible
Selected, $G = 1200$ s	queue-length: $B_0 = 0$, $\eta = 0$, $\gamma = 16k/4$ s; worst-cell gap 0.7283, worst-cell harm 145.3 s against 600 s, 95 of 111 feasible
Per-shape winners	constant: $B_0 = 329.671, 981.126, 2,029.958 k/4$ s at the three promises; age-relative: $(60, 0.50)$, $(120, 0.75)$, $(360, 0.75)$; queue-length: $\gamma = 16k/4$ s with $B_0 = 0, 120k/4$ s, 0
Comparators	As named in Table 3. Fixed(G) is the equal-promise budget $B_{\max}$, which the harm rule rejects at every promise; Skip(G) takes the largest dispatch-charged count (2) allows at that G and that cell's k

8.2. What Was Selected on What

Three statements have to precede the numbers. First, the guard family and a coarse grid over (B_0, η) were explored on the evaluation trace before any validation-only selection existed, so the selection reported above chooses inside a family that had already been

narrowed on the trace this section reports. Second, nothing here is held out. The validation pool is the development terms minus the last one, which is held out of the selection and not of the development process; the last development term on its own cannot load the pool, since it would need 190 copies of its 8 class-terms, and it is therefore not evaluated separately. We do not describe any figure in this section as out-of-sample, validated on test data, or evidence of generalisation.

Third, every development number in this section comes from the released package: the predictors are refitted deterministically from the recorded configuration, and the simulator carries arrival instants, service times and budget arithmetic in exact integer microseconds, so a rerun reproduces the tables byte for byte. Earlier exploratory runs of the same experiments used a floating-point clock and a stored copy of the predictions; where both exist they agree to the first decimal of a wait and differ in the second, and we print the package's values throughout.

The three grids, the feasibility rule, the objective and the tie-break were fixed before the grids were run, and the three shapes are searched at comparable density rather than one being given a finer grid than the others. Two properties of the outcome are worth stating here rather than in the discussion. The winning shape is not the same at every promise: the age-relative shape wins the joint comparison at $G = 300$ and 600 s and the queue-length shape at $G = 1200$ s, where it reaches worst-cell gap 0.7283 at worst-cell harm 145.3 s against 483.9 s for the best age-relative point. And the rule is a worst-cell rule, so a single cell can decide it: dropping the one validation cell that decides the $G = 600$ choice moves that choice to $B_0 = 15k/4$ s, $\eta = 0.90$, with worst-cell gap 0.7237. Five overlays are five week-shifts of the same class-terms and are not five independent tests. The sealed terms will be opened once after the protocol freeze and reported in the same tables as an additional column, with the parameters unchanged.

8.3. Question 2: What to Rank By

Table 7 gives the unguarded policies at the three loads. Ranking by expected cost is ahead of the log-scale score at the two higher loads and unresolved at the lightest: the paired difference in gap closed is -0.000 $[-0.016, 0.013]$ at $\rho = 0.5$, $+0.012$ $[0.004, 0.019]$ at $\rho = 0.8$ and $+0.019$ $[0.011, 0.024]$ at $\rho = 1.0$. The control for refit noise, the same log-scale estimand refitted from scratch, moves the gap by $+0.001$ $[-0.002, 0.002]$, -0.000 $[-0.001, 0.001]$ and -0.000 $[-0.001, 0.000]$, so the advantage is larger than the noise of refitting and is not produced by it; that control and the sequence state come from the earlier development study and are the two rows of Table 7 the package does not produce. The sequence state of Section 4 does not translate into a consistent gain and we do not claim it as an improvement.

Scores other than a conditional mean were fitted on the same features and run through the same simulator (Table 8). A predicted quantile loses at every load, and the loss grows as the quantile falls, which is what a scheduler that needs an additive quantity rather than a threshold should do; squared error on raw seconds also loses at every load, so it is the change of estimand and not the change of scale that produces the difference. A two-class rule that sends predicted-heavy jobs behind predicted-light ones and runs FCFS inside each class reaches a deadline-window p99 of 64.05 s at $\rho = 0.8$ on the first overlay, against 43.86 s for the full ordering on the same overlay, so most of the benefit comes from the fine ordering inside the light class rather than from separating the two classes.

The two kinds of error do not cost the same. Replacing the largest 1% of underestimates by the true cost, and leaving the rest of the score alone, moves the deadline-window p99 at $\rho = 1.0$ from 66.57 s to 45.01 s on the first overlay and from 65.02 s to 45.32 s on the second, which is 0.91 of the distance to the SJF reference in both. Repairing the largest 1%

Table 7. Unguarded policies, five overlays. Waits in seconds; gap and reduction on the deadline-window p99, with 95% paired week-block intervals.

Load	Policy	p99 _{dl}	Gap closed	Red. (%)	Mean
$\rho = 0.5, k = 7.8$	FCFS	27.64	0.000	0.0	0.719
	SJF	10.59	1.000	62.0 [58.7, 66.4]	0.228
	SPJF-log	14.71	0.754 [0.722, 0.777]	46.7 [43.9, 50.1]	0.296
	SPJF-log, refitted ^a	14.72	0.753 [0.722, 0.776]	46.7 [44.0, 50.0]	0.295
	SPJF-E	14.68	0.754 [0.725, 0.776]	46.7 [43.9, 49.9]	0.292
	SPJF-E, GRU state ^a	14.87	0.741 [0.707, 0.767]	45.9 [43.2, 48.8]	0.290
$\rho = 0.8, k = 5$	FCFS	119.35	0.000	0.0	4.160
	SJF	33.55	1.000	71.8 [67.0, 75.1]	0.959
	SPJF-log	44.36	0.873 [0.852, 0.887]	62.7 [57.5, 66.2]	1.252
	SPJF-log, refitted ^a	44.41	0.873 [0.851, 0.886]	62.7 [57.4, 66.2]	1.251
	SPJF-E	43.33	0.885 [0.862, 0.899]	63.6 [58.1, 67.4]	1.232
	SPJF-E, GRU state ^a	42.82	0.891 [0.870, 0.904]	64.0 [58.5, 67.7]	1.230
$\rho = 1.0, k = 4$	FCFS	273.50	0.000	0.0	10.270
	SJF	43.63	1.000	84.0 [79.6, 86.8]	1.696
	SPJF-log	67.17	0.897 [0.872, 0.914]	75.4 [70.2, 78.6]	2.354
	SPJF-log, refitted ^a	67.36	0.896 [0.871, 0.913]	75.3 [70.1, 78.5]	2.352
	SPJF-E	62.79	0.916 [0.892, 0.930]	77.0 [71.6, 80.3]	2.294
	SPJF-E, GRU state ^a	62.71	0.917 [0.895, 0.929]	77.0 [71.7, 80.3]	2.302

^a These two rows are not produced by the released package, which fits only the expected-cost and log-scale scores. They come from the earlier development study (*prechecks/main_v3*), which used a floating-point clock and a stored copy of the predictions, and are printed at the values that study reported; the remaining rows differ from it in the second decimal of a wait.

Table 8. Ranking scores fitted on one feature set and run through one simulator: fraction of the deadline-window gap closed, five overlays, and the paired difference against the log-scale score.

Score	$\rho = 0.5$	$\rho = 0.8$	$\rho = 1.0$	Difference at $\rho = 1.0$
SPJF-log	0.7528	0.8728	0.8967	—
SPJF-E	0.7515	0.8847	0.9159	+0.0191 [+0.0111, +0.0238]
Expected cost, gamma hurdle	0.7404	0.8826	0.9141	+0.0174 [+0.0082, +0.0223]
Probability of heavy	0.7476	0.8821	0.9122	+0.0155 [+0.0066, +0.0199]
Predicted 99th percentile	0.7245	0.8742	0.9005	+0.0038 [−0.0050, +0.0097]
Predicted 90th percentile	0.7024	0.8470	0.8755	−0.0212 [−0.0297, −0.0156]
Predicted median	0.7071	0.8196	0.8402	−0.0565 [−0.0684, −0.0463]
Squared error, raw seconds	0.7197	0.8585	0.8873	−0.0094 [−0.0194, −0.0043]

of overestimates instead moves it to 60.37 s and 59.72 s, which is 0.26 and 0.24 of that distance. At 0.1% the two figures are 0.47 and 0.04. Treating a heavy job as light and letting it run first is the expensive mistake, by about an order of magnitude, which is the argument of Section 5.1 for fitting an estimand rather than minimising a symmetric loss.

8.4. Question 3: What the Guarantee Costs and Buys

Table 9 reports the guarded policies at three promises and three loads, and Figure 3 plots the gap closed against the promise. The cost of the promise depends on the promise and on the load. At $\rho = 1.0$, Guard(600) closes 0.801 of the FCFS-to-SJF gap against 0.916 for the unguarded policy, so it gives up 0.115 of the gap at a promise of ten job-limits, about an eighth of what the unguarded ordering buys; at $G = 1200$ s the cost is 0.087 and at $G = 300$ s, five job-limits, 0.438. These three are differences of the point values in Table 9; the package's paired intervals are computed between guarded policies, which is where the comparisons below sit.

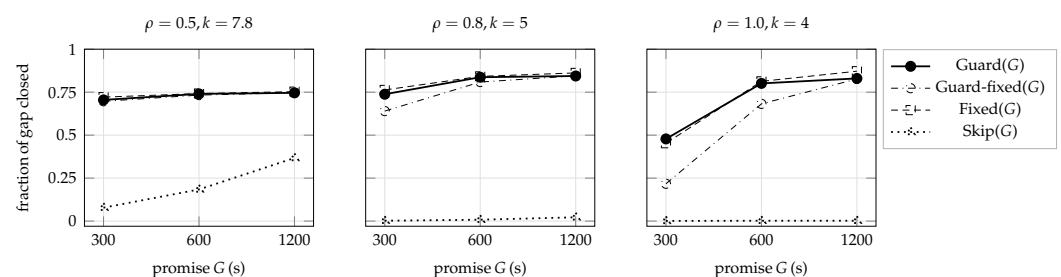


Figure 3. What each budget shape keeps of the FCFS-to-SJF gap as the promised excess G is relaxed. Every point is a “Gap closed” cell of Table 9; the intervals are in that table and are omitted here. All four policies carry the same promise at a given G , since the promise depends on the cap alone. The gap between Guard(G) and its constant-shape ablation opens where the promise is tight and the load is high and closes elsewhere; counting positions rather than work (Skip) collapses at the two higher loads; Fixed(G), which spends the whole cap, buys its gap with harm the selection rule rejects.

What the shape of the budget is worth.

The promise is the cap's alone, so the three shapes are compared at a fixed promise and the question is only how much of the base policy's benefit each keeps and who pays for it. The shape matters where the promise is tight and the load is high, and hardly at all elsewhere. At $G = 600$ s the constant shape trails the joint winner by 0.007 [0.003, 0.012], 0.028 [0.020, 0.038] and 0.119 [0.074, 0.176] of the gap as the load rises through the three levels, each interval clear of zero; at $G = 300$ s the same three differences are 0.008 [-0.002, 0.015], 0.099 [0.056, 0.157] and 0.262 [0.202, 0.288]; and at $G = 1200$ s they are -0.001, -0.001 and 0.003, none of the three intervals excluding zero. At a loose promise and a light load the shape is not doing measurable work.

The other axis is harm. At $G = 600$ s the queue-length shape reaches within 0.008 of the joint winner's gap at every load while its worst-overlay harm is 106.0, 117.7 and 123.4 s against the winner's 237.0, 319.6 and 339.9 s, under half at every load. That is also why the rule picks the queue-length shape at $G = 1200$ s, where the three shapes are within 0.005 of one another on the validation objective and it has the smallest worst harm. A shape moves where the promise is spent more than it moves how much is kept.

Fixed(G), which spends the whole cap, is reported for reference and is not a candidate: its worst-overlay harm is 181.8, 195.4 and 208.2 s at $G = 300$ s against a 150 s limit, 237.0, 447.1 and 465.5 s at $G = 600$ s against 300 s, and 237.0, 469.2 and 708.8 s at $G = 1200$ s against 600 s, so the selection rule rejects it at all three promises. It is also ahead of the joint

Table 9. The guard at three promises, its three budget shapes, and the comparators. Guard(G) is the shape the selection rule picks over all three grids; Guard-fixed, Guard-age and Guard-queue are the winners inside the constant, age-relative and queue-length grid, so one of them repeats Guard(G) at each promise. Fixed(G) spends the whole cap at the same promise and is infeasible under the harm rule. Budgets scale as $k/4$ and the skip count is the largest integer with $(N + 2k - 2)L/k \leq G$, so both differ with the cell's k : at $\rho = 0.5$ the four overlays at $k = 8$ run $N = 26/66/146$ and the fifth, at $k = 7$, runs $23/58/128$, which is what the column prints. Excess, harm and worst heavy wait are the worst over the five overlays, the rest are means.

Load	Policy	Promise	p99 _{dl}	Gap closed	Red. (%)	Max exc.	Harm	Fired (%)
$\rho = 0.5, k = 7.8$; FCFS p99 _{dl} = 27.64 s, SPJF-E 14.68 s with max excess 1,241.4 s								
	Guard(300)	300	15.58	0.705 [0.678, 0.724]	43.7	192.5	132.1	1.35
	Guard-fixed(300)	300	15.75	0.697 [0.673, 0.717]	43.2	143.0	132.1	2.12
	Guard-age(300)	300	15.58	0.705 [0.678, 0.724]	43.7	192.5	132.1	1.35
	Guard-queue(300)	300	15.27	0.723 [0.697, 0.742]	44.8	193.4	94.3	21.60
	Fixed(300)	300	15.30	0.721 [0.696, 0.738]	44.7	193.0	181.8	0.77
	Skip(300), $N = 23$	300	26.38	0.078 [0.052, 0.107]	4.8	62.1	59.6	45.41
	Guard(600)	600	14.97	0.740 [0.713, 0.760]	45.8	492.2	237.0	0.17
	Guard-fixed(600)	600	15.09	0.733 [0.706, 0.752]	45.4	298.7	237.0	0.36
	Guard-age(600)	600	14.97	0.740 [0.713, 0.760]	45.8	492.2	237.0	0.17
	Guard-queue(600)	600	14.88	0.744 [0.718, 0.765]	46.1	504.5	106.0	0.16
	Fixed(600)	600	14.91	0.742 [0.715, 0.763]	46.0	493.3	237.0	0.11
	Skip(600), $N = 58$	600	24.64	0.184 [0.135, 0.233]	11.5	100.2	63.3	37.26
	Guard(1200)	1200	14.84	0.746 [0.719, 0.768]	46.2	907.8	94.3	22.01
	Guard-fixed(1200)	1200	14.83	0.747 [0.719, 0.768]	46.2	557.8	237.0	0.06
	Guard-age(1200)	1200	14.73	0.751 [0.723, 0.773]	46.5	1,049.1	237.0	0.01
	Guard-queue(1200)	1200	14.84	0.746 [0.719, 0.768]	46.2	907.8	94.3	22.01
	Fixed(1200)	1200	14.70	0.753 [0.724, 0.775]	46.6	1,077.0	237.0	0.00
	Skip(1200), $N = 128$	1200	21.62	0.367 [0.288, 0.432]	22.8	102.3	67.6	28.80
$\rho = 0.8, k = 5$; FCFS p99 _{dl} = 119.35 s, SPJF-E 43.33 s with max excess 3,044.5 s								
	Guard(300)	300	55.94	0.738 [0.697, 0.761]	53.0	209.7	114.1	11.57
	Guard-fixed(300)	300	64.44	0.640 [0.543, 0.699]	45.9	171.6	151.1	20.62
	Guard-age(300)	300	55.94	0.738 [0.697, 0.761]	53.0	209.7	114.1	11.57
	Guard-queue(300)	300	54.69	0.753 [0.713, 0.777]	54.0	210.7	117.6	17.53
	Fixed(300)	300	53.83	0.763 [0.721, 0.785]	54.8	210.9	195.4	10.75
	Skip(300), $N = 17$	300	119.20	0.002 [-0.001, 0.009]	0.1	64.2	58.8	80.53
	Guard(600)	600	47.47	0.837 [0.815, 0.851]	60.1	507.8	319.6	1.31
	Guard-fixed(600)	600	49.85	0.809 [0.786, 0.824]	58.1	315.4	285.7	4.72
	Guard-age(600)	600	47.47	0.837 [0.815, 0.851]	60.1	507.8	319.6	1.31
	Guard-queue(600)	600	47.69	0.834 [0.814, 0.847]	59.9	508.5	117.7	1.35
	Fixed(600)	600	47.03	0.842 [0.824, 0.853]	60.5	502.6	447.1	1.16
	Skip(600), $N = 42$	600	118.69	0.008 [-0.002, 0.026]	0.6	97.5	63.5	74.02
	Guard(1200)	1200	46.85	0.844 [0.823, 0.857]	60.6	1,104.9	117.6	9.93
	Guard-fixed(1200)	1200	46.76	0.845 [0.826, 0.857]	60.7	581.8	469.2	0.86
	Guard-age(1200)	1200	45.16	0.864 [0.844, 0.876]	62.0	1,102.9	370.5	0.29
	Guard-queue(1200)	1200	46.85	0.844 [0.823, 0.857]	60.6	1,104.9	117.6	9.93
	Fixed(1200)	1200	45.32	0.862 [0.842, 0.874]	61.9	1,106.3	469.2	0.22
	Skip(1200), $N = 92$	1200	117.48	0.022 [0.004, 0.074]	1.6	162.5	83.6	66.10
$\rho = 1.0, k = 4$; FCFS p99 _{dl} = 273.50 s, SPJF-E 62.79 s with max excess 5,904.5 s								
	Guard(300)	300	163.40	0.478 [0.387, 0.602]	40.2	229.1	134.8	28.64
	Guard-fixed(300)	300	223.69	0.217 [0.151, 0.359]	18.2	165.1	158.4	41.50
	Guard-age(300)	300	163.40	0.478 [0.387, 0.602]	40.2	229.1	134.8	28.64
	Guard-queue(300)	300	169.22	0.453 [0.368, 0.607]	38.1	231.4	144.8	30.99
	Fixed(300)	300	169.29	0.453 [0.369, 0.619]	38.1	222.6	208.2	28.33
	Skip(300), $N = 14$	300	273.14	0.001 [-0.001, 0.003]	0.1	81.3	81.3	90.36
	Guard(600)	600	89.24	0.801 [0.764, 0.842]	67.3	523.7	339.9	7.15
	Guard-fixed(600)	600	116.75	0.682 [0.601, 0.764]	57.3	325.4	292.9	17.50
	Guard-age(600)	600	89.24	0.801 [0.764, 0.842]	67.3	523.7	339.9	7.15
	Guard-queue(600)	600	91.09	0.793 [0.758, 0.833]	66.6	515.8	123.4	7.08
	Fixed(600)	600	86.53	0.813 [0.778, 0.851]	68.3	515.8	465.5	7.23
	Skip(600), $N = 34$	600	272.98	0.002 [-0.004, 0.006]	0.2	112.4	82.4	86.25
	Guard(1200)	1200	82.67	0.830 [0.801, 0.857]	69.7	1,110.4	144.8	6.21
	Guard-fixed(1200)	1200	83.32	0.827 [0.798, 0.860]	69.5	576.5	522.7	5.76
	Guard-age(1200)	1200	72.52	0.874 [0.851, 0.893]	73.4	1,117.3	453.3	0.81
	Guard-queue(1200)	1200	82.67	0.830 [0.801, 0.857]	69.7	1,110.4	144.8	6.21
	Fixed(1200)	1200	72.94	0.872 [0.846, 0.893]	73.3	1,112.9	708.8	0.73
	Skip(1200), $N = 74$	1200	273.08	0.002 [-0.009, 0.013]	0.1	169.7	105.0	80.69

winner on the gap closed in eight of the nine cells, by up to 0.042, which is what buying gap with harm looks like; we make no claim that any shape beats it on the gap at equal promise.

Counting positions rather than work buys almost nothing on this load (Figure 3, dotted). The reason is in the cost distribution rather than in the mechanism: overtakers on this trace are overwhelmingly shorter than a second, so a count sized for the worst case NL is exhausted long before the work it permits is, and the baseline forces most dispatches back to the queue head. It is charged at dispatch and given the largest count (2) allows at each promise, 34 rather than 30 passes at $k = 4$ and $G = 600$ s; charging at completion instead changes the gap closed by at most 0.015, and by at most 0.001 at the two higher loads.

Corrupted scores separate what the guard does from what it does not do (Table 10, Figure 4). Under a reversed order, a random order, or a score that calls the truly longest 1% the shortest, the unguarded policy delays some job by more than 6,000 s beyond FCFS at full load, and Guard(600) holds every job inside its promise. The deadline-window p99 is not rescued: it stays at 546.5, 261.9 and 543.6 s against 273.50 s for FCFS, which is worse than FCFS for two of the three. This is the guarantee behaving as stated. It is per job and additive, so a guarded run may have a worse quantile than FCFS, by at most G , and what it removes is the worst single job rather than the shape of the tail.

Table 10. Deliberately corrupted scores, unguarded and under Guard(600). Waits and excesses in seconds.

Load	Score	p99 _{dl}		Max excess		Harm guarded
		unguarded	guarded	unguarded	guarded	
$\rho = 0.5$	reversed	42.72	43.39	1,240.1	497.1	287.1
	random	16.82	17.12	1,307.7	489.9	324.0
	longest as shortest	38.24	38.71	1,305.8	497.1	273.4
$\rho = 0.8$	reversed	228.12	232.41	3,387.9	513.5	323.4
	random	78.52	88.39	3,319.4	504.5	352.3
	longest as shortest	212.54	222.52	3,292.0	516.1	306.0
$\rho = 1.0$	reversed	557.44	546.48	6,295.7	534.8	309.1
	random	194.78	261.93	6,235.5	520.7	332.9
	longest as shortest	523.55	543.63	6,107.8	534.7	324.6

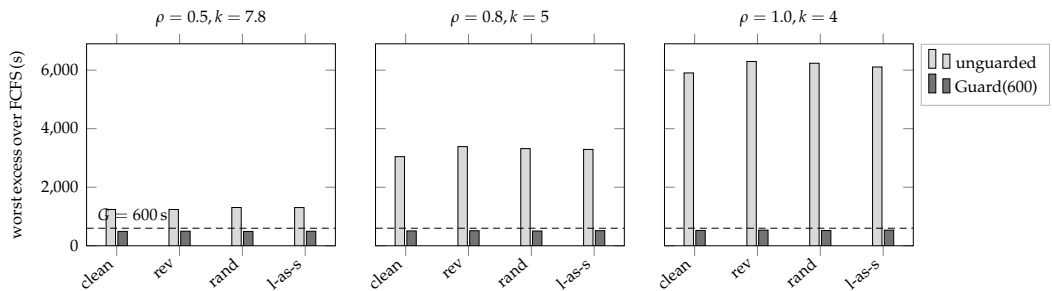


Figure 4. The worst single job, with and without the guard, under the expected-cost score (“clean”) and the three corrupted scores of Table 10: reversed (rev), random (rand) and truly-longest-called-shortest (l-as-s). The dashed line is the promise $G = 600$ s. Sources: the clean pair is the block header and the Guard(600) row of Table 9; the three corrupted pairs are the “Max excess” columns of Table 10. The unguarded bar grows with load and with corruption; the guarded bar does not, which is the content of Theorem 3 and not a property of the trace.

8.5. The Bounds on the Traces

Every guarded run asserts the per-job bound of Section 6 for each job separately, inside the simulation rather than afterwards on summary statistics. Across the 315 policy-

by-cell runs reported here, that is 5,554,949,400 per-job checks with 0 violations, and the single-server configuration adds 21 runs and 67,396,287 checks, also with 0 violations. An independently written simulator, which shares no code with ours, reproduces the runs job for job on 17,634,760 jobs and 22 policies with 0 differing jobs, and asserts the same bounds on its own waiting times over 5.7 billion checks with 0 violations; that cross-check was run against the earlier development pipeline, whose dispatch sequence the package reproduces.

The ratio of the realised worst excess to the bound falls as k grows. It reaches 0.9950 at $k = 1$, 0.9311 at $k = 4$, 0.9219 at $k = 5$, 0.8975 at $k = 7$ and 0.8850 at $k = 8$. This is a statement about these traces and not about the constant: Section 6 shows $(3 - 2/k)L$ to be the exact supremum at every k , but the family that approaches it is a geometric cascade that needs hundreds of jobs arranged against the wrapper, and nothing in a replayed workload arranges them. The finite-skip bound, by contrast, is attained exactly on small instances, at a ratio of 1.0000.

The identity, measured job by job.

The net-overtake identity had been attacked on small synthetic instances but never measured on the trace this paper runs on, so we measured it. Table 11 reports the residual $D_i = k(W_P[i] - W_{FCFS}[i]) - (In_i - Out_i)$ on overlay 0 at the three load levels, under SPJF-E, Guard(600) and the reversed score, with FCFS on the same input as the reference schedule in every cell. Recovering In_i and Out_i needs the dispatch sequence, which the guard kernel does not expose, so the kernel and the independent reference simulator were each copied verbatim and given recorders that add no branch and no state update; the two agree on the dispatch sequence of every job of 3,000 random instances, the recording kernel reproduces the production kernel's waits element for element inside every cell, and In_i and Out_i are carried in exact integer microseconds, the currency the guard itself meters.

Theorem 1 is asserted on every job of every cell: 0 violations over 1,428,415,560 job-checks, the 27 policies of this overlay at the three loads, of which Table 11 prints nine. The residual stays far from its ceiling. The largest $|D_i|$ among the printed cells is $6.788L$, at $k = 8$, where the bound $2(k - 1)L$ is $14L$, and across the nine cells the maximum sits between 0.485 and 0.514 of the bound, which is about $(k - 1)L$; over all 81 cells it never exceeds 0.517. That is where a search that does not build the cascade of Theorem 2 stops, so the measurement says the workload does not attain the constant, not that the constant is loose. Same-phase bookkeeping, the mechanism behind the degenerate extremal instances of Remark 1, accounts for at most 0.36% of the total In in any cell, so the executed-work convention would not change the picture here. The net-overtake term carries the per-job excess almost entirely: $(In_i - Out_i)/k$ explains $R^2 = 0.983$ to 0.9996 of the variance of $W_P[i] - W_{FCFS}[i]$, with a worst per-job error of 44.9 to 50.9 s where the theorem permits $(2 - 2/k)L$, that is 90 s at $k = 4$ and 105 s at $k = 8$. This is one overlay of development data, and the residual is a property of a schedule rather than a statistic, so no interval is attached to it.

8.6. A Single Server

At $k = 1$ the additive constant $(3 - 2/k)L$ is exactly L , the identity of Section 6 holds with no error term, and the separation between budget shapes is largest (Table 12). Guard(600) and Fixed(600) are statistically indistinguishable on the gap closed and differ by a factor of 3.41 in harm; the corresponding factors are 3.01 at $G = 300$ s and 12.18 at $G = 1200$ s. The realised excess reaches 595.5 s against the promised 600 s, and under the worst corrupted score 597.0 s, so at one server the promise is close to being spent.

Table 11. The residual D_i of Theorem 1 on overlay 0, 17,634,760 jobs per cell. Ratio is $\max |D_i|$ over the bound $2(k - 1)L$, which is $14L$, $8L$ and $6L$ at the three loads; R^2 and the worst error are for $(In_i - Out_i)/k$ against the realised excess over FCFS.

Load	Policy	$\max D_i /L$	Ratio	$D_i = 0$	R^2	worst err. (s)
$\rho = 0.5, k = 8$	SPJF-E	6.788	0.485	95.6%	0.985	50.9
	Guard(600)	6.788	0.485	95.6%	0.983	50.9
	Reversed score	6.788	0.485	95.6%	0.996	50.9
$\rho = 0.8, k = 5$	SPJF-E	3.922	0.490	87.2%	0.996	47.1
	Guard(600)	3.922	0.490	87.2%	0.995	47.1
	Reversed score	3.881	0.485	87.5%	0.999	46.6
$\rho = 1.0, k = 4$	SPJF-E	3.082	0.514	81.1%	0.998	46.2
	Guard(600)	3.082	0.514	81.1%	0.998	46.2
	Reversed score	2.994	0.499	81.7%	1.000	44.9

The single-server configuration also shows the cost of choosing a shape on one set of cells. The parameters were selected on the 15 multi-server validation cells, and $k = 1$ is not among them. At $G = 300$ and 600 s the joint winner transfers: it closes 0.746 and 0.854 of the gap, against 0.543 and 0.791 for the constant shape. At $G = 1200$ s it does not: the queue-length shape that wins the joint comparison on the multi-server cells closes 0.701 here, while the age-relative shape closes 0.895 and the constant shape 0.865 . We report the number the pre-stated rule produces and do not re-tune it on this cell; Section 9 states what it means.

Table 12. Single server, busy-hour $\rho = 0.80$, 3,209,347 jobs. Waits and excesses in seconds. The parameters are the ones the rule selected on the multi-server validation cells; this configuration is not among them.

Policy	Promise	p99 _{dl}	Gap closed	Max exc.	Harm	Fired (%)
FCFS	—	221.27	0.000	0.0	0.0	100.00
SJF on true sizes	—	45.32	1.000	2,054.5	777.9	0.00
SPJF-E	—	60.91	0.911 [0.854, 0.933]	2,520.3	656.4	0.00
Guard(300)	300	89.96	0.746 [0.571, 0.829]	297.9	81.8	8.17
Guard-fixed(300)	300	125.68	0.543 [0.207, 0.718]	140.3	119.5	17.09
Guard-age(300)	300	89.96	0.746 [0.571, 0.829]	297.9	81.8	8.17
Guard-queue(300)	300	105.15	0.660 [0.532, 0.715]	297.9	53.9	29.15
Fixed(300)	300	83.18	0.785 [0.584, 0.865]	297.9	246.3	6.33
Skip(300), $N = 5$	300	220.39	0.005 [-0.002, 0.019]	95.1	81.8	73.17
Guard(600)	600	71.07	0.854 [0.780, 0.895]	595.5	165.5	2.53
Guard-fixed(600)	600	82.10	0.791 [0.602, 0.868]	303.0	246.3	6.16
Guard-age(600)	600	71.07	0.854 [0.780, 0.895]	595.5	165.5	2.53
Guard-queue(600)	600	91.52	0.737 [0.640, 0.812]	582.0	81.8	3.43
Fixed(600)	600	68.48	0.868 [0.783, 0.906]	593.4	563.6	2.29
Skip(600), $N = 10$	600	219.89	0.008 [-0.004, 0.037]	100.9	83.2	66.98
Guard(1200)	1200	98.00	0.701 [0.640, 0.742]	1,157.4	53.9	26.15
Guard-fixed(1200)	1200	69.03	0.865 [0.789, 0.905]	564.2	509.3	2.57
Guard-age(1200)	1200	63.80	0.895 [0.835, 0.920]	1,182.3	366.4	0.68
Guard-queue(1200)	1200	98.00	0.701 [0.640, 0.742]	1,157.4	53.9	26.15
Fixed(1200)	1200	63.19	0.898 [0.837, 0.923]	1,180.4	656.4	0.45
Skip(1200), $N = 20$	1200	217.64	0.021 [-0.008, 0.084]	168.0	97.7	58.67

8.7. Other Platforms

Table 13 places the main trace beside five others under one format. The first three columns describe the workload, the next two what ranking by prediction does to it without a guard, and the last two what the guard keeps and what it caps at a promise of five

job-limits. Excesses are expressed in units of the per-job limit L , which is the unit the guarantee is written in.

Table 13. The same method on six workloads. Tail share is the fraction of executed work carried by the longest 1% of jobs; AUROC is for the heavy class from arrival-time information; excesses are in units of the per-job limit L of that trace.

Trace and load	Tail share	AUROC	FCFS p99/ L	Gap SPJF-E	Unguarded max exc.	Guard at $G \approx 5L$ gap kept / max exc.
CodeBench, busy-hour $\rho = 1.06$	0.375	0.928	4.39	0.916	98.4 L	0.478 / 3.82 L
ACcoding, drawn arrivals, $\rho = 0.8$	0.122	0.927	0.021	0.980	20 L	0.974 / 2.21 L ^a
Serverless, busy-hour $\rho = 1.0$	0.443	0.985	4.22	0.858	10.07 L	0.710 / 2.52 L
CI pool A, $k_{\text{eff}} = 156$ of 173	0.034	0.651	1.78	n/a ^b	11.53 L	0.735 ^b / 2.64 L
CI pool B, $k_{\text{eff}} = 96$ of 105	0.052	0.997	0.373	0.958	0.34 L	0.958 / 0.34 L
Compute farm, window $\rho = 0.95$	0.372	0.890	0.136	−4.111	0.85 L	−4.346 / 0.83 L

^a ACcoding arrival instants are drawn, not recorded, so every scheduling figure in this row is a statement about the ordering mechanism and not about waiting on that platform; its budgets were set at fixed multiples of L rather than from a promise, and the entry shown is the largest one tested, $B = 10L$. ^b On CI pool A, SJF is itself worse than FCFS at the p99, so the p99 gap has no usable denominator; the guard entry is the mean-wait gap. ^c The CI capacities are fitted, not recorded, and the two scheduling columns and the FCFS p99 of these rows exist only at the whole-window fit ($k_{\text{eff}} = 156$ of 173 and 96 of 105), so they are printed at it. What supports them is the held-out refit of Section 7, which recovers the same capacity to within a few machines on chronological splits of each window and reproduces the recorded waits on the untouched part, with the p90 the one quantile that does not survive. The tail share and the AUROC do not depend on the capacity.

Two conditions decide whether the method is worth applying, and each has a trace here that fails it. The first is that the per-job limit L must be small next to the waits the operator cares about. The smallest promise the guarantee family can express is $(3 - 2/k)L$, close to $3L$, so a workload whose FCFS p99 wait is below that ratio cannot be given a promise finer than the wait it is about. The ratio is 4.39 on CodeBench and 4.22 on the serverless trace, 1.78 on CI pool A and 0.136 on the compute farm, whose jobs run for days: in that window its FCFS p99 wait is 18.7 h against a per-job limit of 493,857 s, or 5.7 days. The second is that costs must be heavy-tailed. On the CI pools the longest 1% of jobs carry 3.4% and 5.2% of the work, and SJF makes the p99 wait worse than FCFS on three of the four windows we ran, so there is no p99 benefit to transmit and the gap statistic has no denominator. Where both conditions hold, the guard keeps 0.48 to 0.71 of the gap and caps the observed excess at 2.5 to 3.8 L at a promise of 5 L .

8.8. Sensitivities

Five variations of the trace construction and of the availability protocol were run through the same pipeline (Table 14). None of them reverses the result and the two that move it move it upwards, so the primary pool is the conservative choice: adding the term whose busiest hour is dominated by runs that outlived their container raises the gap, and the opposite reading of the timestamp, in which the stamp is receipt rather than completion, gives 0.926 against 0.895. Merging or dropping the groups of runs by one user that end in the same second moves the gap closed by 0.017 and 0.003 at full load, so the stretching between concurrent runs of one user is not what produces the result, and delaying the visibility of a completed job’s outcome costs prediction quality and not much of it (Section 4). The guard’s benefit is the quantity most sensitive to the construction, ranging from 0.442 to 0.843 of the gap at a fixed budget of 600 s at full load.

Two further limits of these runs are stated where they belong. The intervals resample whole weeks of one overlay construction, so they cover time and not the choice of terms, and five overlays are five week-shifts of the same 40 class-terms rather than five platforms. The runs compare scheduling policies on a common set of enqueue instants and do not charge the cost of computing features and predictions; Section 4 measures that cost separately, and Section 9 states what it would take to claim an end-to-end result.

Table 14. Trace-construction sensitivities at busy-hour $\rho = 1.0$, three overlays, fraction of the deadline-window gap closed.

Variant	k	FCFS $p_{99_{dl}}$	Unguarded	Guard, budget 600 s
Primary pool (overlays 0–2)	4.000	—	0.895 [0.864, 0.916]	0.442 [0.345, 0.631]
Earlier term added	4.667	95.41	0.945 [0.885, 0.965]	0.843 [0.715, 0.880]
Co-ending runs merged	4.000	228.19	0.912 [0.892, 0.924]	0.603 [0.445, 0.740]
Co-ending runs dropped	4.000	209.30	0.898 [0.879, 0.914]	0.608 [0.471, 0.753]
Opposite timestamp reading	4.000	268.75	0.926 [0.904, 0.939]	0.545 [0.422, 0.699]

9. Limitations and Threats to Validity

The shared pool is our design.

CodeBench gives each logged-in student a private container, so on the real platform one job never waits behind another and there is no shared judging queue [13]. We use its logs as a demand trace: the arrival instants and the per-job costs are recorded; the server pool, the queue and the scheduler are ours. Section 7 sets out what supports that design, that a fixed pool of judge hosts on one queue is a standard deployment [2,3] and that pooling replaces 1,636 concurrent containers with 14 servers on the same demand [62]. What it cannot support is any statement about waiting on the real platform, the pooled side being simulated throughout. The container counts belong to the 44-fold overlay rather than to a deployment, and a container also serves the student’s editing session, so the ratio is not a cost saving.

Open-loop replay.

Arrivals come from the log at their recorded instants and do not react to our scheduler. Feitelson states the objection at its strongest, that “logged workloads actually contain a ‘signature’ of the logged system” [63]; a trace-driven simulation issues requests irrespective of system state, so a scheduler is neither rewarded with the work it would attract nor punished by the users it would drive away [64]. We cannot restore that feedback, our logs carrying no session boundaries, no think times and no resubmission semantics, but its direction is known: a closed loop returns work to a service that answers quickly, so the omission suppresses the benefit of the policies that shorten waits, and our reported gains are a lower bound. Replaying into a differently sized pool is established practice [65], and the width coupling that distorts parallel-job replays does not arise here, a job occupying one server whatever k is. One choice is deliberate. We raise load by superposing whole class-semesters aligned on weekday and clock time rather than by compressing inter-arrival times, which Feitelson notes shrinks the daily load cycle along with the gaps [63].

The guarantee is denominated in L .

Every constant in Section 6 is a multiple of the per-job time limit. The additive constant $(3 - 2/k)L$ is an exact supremum rather than an artefact of the proof, it grows with k from L at a single server towards $3L$, and an excess of at least L is forced on some job for any non-preemptive work-conserving policy that reads a service time only at completion, so no mechanism of this family can promise less than $\Omega(L)$. We tested the obvious escape and it fails. Kill-and-restart tiers runs every job for a short first quantum τ and restarts the survivors, discarding the killed attempt; on n jobs of length L arriving together it leaves some job with an excess of $n\tau/k - L$ over first-come first-served, and that excess is wasted work rather than overtaking, so no budget removes it. The light-job constant does not improve either, a second-tier piece being still a non-preemptive block of length L :

holding τ fixed and varying L , the worst light-job additive constant we measure grows as about $1.1L$ and is flat in τ . The mechanism is TAGS [66] with the dedicated hosts removed.

Two applicability conditions, each with a trace that fails it.

The first is that L be comparable to the waits that matter. On the Intel Netbatch compute farm [15,16] the service cap is 5.7 days while the first-come first-served p99 wait in the worst window we build is 18.7 hours, so the smallest expressible promise is about 17 days and the guard never fires at any admissible budget; predicted-cost ordering also makes the p99 wait worse than first-come first-served there under sustained overload. The second is that the cost distribution be heavy-tailed. On the two Firefox CI hardware pools [57,58] the longest 1% of jobs carry 3.4% and 5.2% of the work, at coefficients of variation 0.61 and 0.87, and there even true-size ordering makes the p99 wait worse than first-come first-served on three of the four windows we ran. Section 8.7 gives both conditions in numbers.

The guarantee is per job, not per quantile.

The bound holds for each job separately and says nothing about a percentile. A guarded run can have a worse deadline-window p99 than the baseline its guarantee is stated against, by as much as the promise itself, and Table 10 shows it doing so under two of the three corrupted scores at the busiest load while the worst single job stays inside the promise. An operator who needs a percentile has to ask for one.

Development data, and what is still sealed.

Every CodeBench number in this paper is development data. The guard family and an early parameter grid were explored on the same overlay the tables report, before the three pre-stated grids were searched on validation overlays that exclude the evaluation semester, so the selection is validation-only inside a family that is not, and no number here is out of sample. The rule is a worst-cell rule, so it is sensitive to which validation cells it sees: at the middle promise, dropping the one cell that decides the choice moves it to a different point of the same grid, and five overlays are five week-shifts of the same class-terms rather than five independent tests. We do not describe any of these parameters as validated. The guarantee does not depend on the parameters being well chosen; only the benefit does. The sealed semesters are opened once, after the method, the parameters and the primary metric have been frozen, and their numbers are reported separately.

The budget shape is selected on one set of cells, and does not always transfer.

The promise holds for any budget under the cap, so nothing about the guarantee depends on the shape. What the shape decides is how much of the base policy's benefit survives, and that we select on 15 multi-server validation cells, which is the whole evidence we have for it. The single-server configuration shows the limit of that evidence. At $G = 1200$ s the queue-length shape wins the joint comparison on those cells and closes 0.701 of the gap at $k = 1$, where the age-relative shape closes 0.895; at the two tighter promises the same choice does transfer. We report the number the pre-stated rule produces rather than re-tuning on the configuration that exposes it, and an operator whose pool is much smaller than the validation pool should re-run the selection rather than take our shape. The three shapes also differ in where the harm falls rather than only in how much gap they keep, so which of them is preferable depends on a judgement about jobs first-come first-served would have started at once, and that judgement is the operator's.

What the recorded costs mean.

The cost we replay is the platform's recorded execution time, a wall-clock duration for one evaluation that includes interpreter start-up. Runs of the same student that overlap stretch one another, and between 38% and 79% of jobs longer than a second end in the same second as another block of that student, so part of the recorded cost is contention we then re-impose inside the pool. How the platform enforces its 60-second limit is documented nowhere we could find. We wrote to the maintainers about both points and had no answer. ACcoding [14] records no submission timestamps, so its arrival process is synthetic. The Firefox CI check, the one trace whose queue waits are themselves recorded, fits an effective capacity and a setup term, both refitted on one chronological part of each window and tested on the other; out of sample the mean wait, the per-hour profile and the per-job ordering survive and the 90th percentile does not, landing up to 21% from the recorded one and systematically low on the larger pool, so the simulated distribution is too steep in its upper middle. The split is chronological inside one 21-day and one 7-day window of two pools, not a different cluster, quarter or workload, and capacity is held constant over the test part although the real machine count varies.

Not studied.

Preemption is outside the model, so nothing here applies to a system that can checkpoint or suspend. The wrapper applies unchanged to a learned or reinforcement-learned base policy, its bound assuming nothing about the base, but we did not test one. Capacity is fixed throughout, so this is not a capacity-planning study. The cost of computing features and running the predictor is not charged in the reported runs, which compare policies at a common set of enqueue instants, and at a median service time of a fraction of a second it is not negligible. Nothing here was deployed.

Conclusion.

We asked whether a non-preemptive shared execution service can be reordered by predicted cost without exposing any single job to an unbounded wait. What the evidence supports is that the currency has to be work rather than positions. A job's delay relative to first-come first-served equals its net overtaken work divided by the number of servers, up to a constant multiple of the time limit, and a budget on that quantity is sufficient and necessary for a per-job guarantee. The wrapper built on it holds every job to a promise the operator states in seconds, whatever the predictor does, at a cost of about an eighth of the gap predicted-cost ordering closes at the busiest load we simulate and at a promise of ten time limits. The method needs a time limit comparable to the waits at issue and a heavy-tailed cost distribution, and we give a workload on which each fails.

Author Contributions: [to be completed by the authors]. The template expects the CRediT roles (conceptualization, methodology, software, validation, formal analysis, investigation, data curation, writing—original draft preparation, writing—review and editing, visualization, supervision, project administration, funding acquisition), each attributed by author initials, followed by "All authors have read and agreed to the published version of the manuscript."

Funding: [to be completed by the authors]. If there is no external funding, the template asks for "This research received no external funding"; otherwise the grant number and the funding body go here.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Five of the six workloads used here are public at the locations cited, and we redistribute none of them. The CodeBench judging logs are published by the Universidade

Federal do Amazonas [13]; the ACcoding dataset is published with its data descriptor [14]; the Open University Learning Analytics dataset is published with its own [59]; the Azure Functions 2021 invocation trace is published in the Azure Public Dataset repository [7], which asks that [8] be cited; and the Intel Netbatch logs are published in the Parallel Workloads Archive [15], which asks that [16] be cited and that the logs be acknowledged to their depositors. The sixth, the continuous-integration trace of the two Firefox hardware worker pools, was collected by us from Mozilla's public, unauthenticated Taskcluster and Treeherder APIs [57,58]; we do not redistribute it, and the collection scripts that rebuild it from those APIs are part of the released code. No data were obtained under licence or under an access agreement, and no personal data were processed beyond the pseudonymous user identifiers the published judging datasets already carry. The simulator, the predictors, the guard implementation and the configuration files that reproduce every table and figure in this paper are to be released with the paper at [repository URL to be completed by the authors].

Conflicts of Interest: [to be completed by the authors]. The template asks for "The authors declare no conflicts of interest" where that holds, and otherwise for the competing interest and for a statement on the role of any funder in the design, collection, analysis or interpretation, in the writing, or in the decision to publish.

Abbreviations

The following abbreviations are used in this manuscript:

AUPRC	Area under the precision–recall curve
AUROC	Area under the receiver operating characteristic curve
CI	Continuous integration
FCFS	First-come first-served
GRU	Gated recurrent unit
MLP	Multilayer perceptron
OULAD	Open University Learning Analytics Dataset
RMSE	Root mean squared error
SJF	Shortest job first
SPJF	Shortest predicted job first
SRPT	Shortest remaining processing time

Appendix A. Proofs of the tightness results

This appendix gives in full the constructions sketched in Section 6. All of them are exact in integer arithmetic; we simulated every one of them and checked every intermediate quantity of every proof job by job, including an independent audit of work conservation and server capacity re-derived from the produced schedule.

Appendix A.1. The workload comparison

Proof of Lemma 1. Put $D = U_A - U_B$. Arrivals add the same jump to both terms, so D is continuous, and between arrivals it changes at rate $-(\min(N_A, k) - \min(N_B, k))$; it is therefore piecewise linear with breakpoints among the finitely many arrival and completion instants of the two runs.

If $U_A(u) > (k-1)L$ then more than $k-1$ jobs are present under A , each carrying at most L , so every server of A is busy and $\min(N_A, k) = k \geq \min(N_B, k)$: D does not increase at u . Fix t and let $S = \{u \leq t : U_A(u) \leq (k-1)L\}$, which is non-empty because U_A vanishes before the first arrival, and put $t_0 = \sup S$. The supremum need not be attained, since U_A jumps upwards at arrivals, so we use the left limit: there are $u_n \uparrow t_0$ with $U_A(u_n) \leq (k-1)L$, hence $U_A(t_0^-) \leq (k-1)L$ and, D being continuous, $D(t_0^-) \leq U_A(t_0^-) \leq (k-1)L$ because $U_B \geq 0$. On $(t_0, t]$ we have $U_A > (k-1)L$, so D does not increase there, and $D(t) \leq (k-1)L$. Exchanging A and B gives the other direc-

tion. That the constant cannot be lowered is the cascade of Appendix A.2, which reaches $(k-1)L(1 - ((k-1)/k)^m)$ after m rounds. $\square$

Proof of the strict inequality in Lemma 1. Keep the notation above, so $D = U_A - U_B$ is continuous and piecewise linear with finitely many pieces. Suppose $D(t) = (k-1)L$ for some t and let t_1 be the smallest such t ; it exists because $\{D = (k-1)L\}$ is closed and $D(0) = 0 < (k-1)L$, using $L > 0$.

Let I be the linear piece immediately to the left of t_1 , which has positive length. Its slope is strictly positive: a negative slope would put $D > (k-1)L$ somewhere on I and a zero slope would put $D = (k-1)L$ on all of I , and either contradicts the minimality of t_1 . The slope of D is $-(\min(N_A, k) - \min(N_B, k))$, so $\min(N_A, k) < \min(N_B, k)$ on I and therefore $N_A \leq k-1$ there.

With at most $k-1$ jobs present and A work-conserving, none of A 's jobs waits, so all of them are in service; hence U_A falls at rate N_A on I , and $U_A \leq (k-1)L$ on I because each of the at most $k-1$ present jobs carries at most L . Taking left limits at t_1 and using the continuity of D ,

$$U_A(t_1^-) - U_B(t_1^-) = (k-1)L, \quad U_A(t_1^-) \leq (k-1)L, \quad U_B \geq 0,$$

which forces $U_B(t_1^-) = 0$ and $U_A(t_1^-) = (k-1)L$. Since $L > 0$, this gives $N_A \geq 1$ on I — with $N_A = 0$ we would have $U_A(t_1^-) = 0$ — so U_A is strictly decreasing on I and $U_A(t_1^-) < \sup_I U_A \leq (k-1)L$, a contradiction. Exchanging A and B gives the other direction. $\square$

Appendix A.2. The cascade

Both tightness results begin from the same construction, which drives the workload gap of Lemma 1 to its ceiling geometrically.

Lemma A1. Fix $k \geq 2$ and $m \geq 1$ and set $L = k^m$. There is an input and a work-conserving policy P such that, at time $T_m = mL$, first-come first-served has completed every job and idles all k servers, while P holds exactly $k-1$ jobs in service, each with remaining service

$$f = \frac{\rho_m}{k-1} = L \left(1 - \left(\frac{k-1}{k}\right)^m\right), \quad \rho_m = (k-1)L \left(1 - \left(\frac{k-1}{k}\right)^m\right),$$

so that $U_P(T_m) - U_{\text{FCFS}}(T_m) = \rho_m \rightarrow (k-1)L$ as $m \rightarrow \infty$. Every ρ_j and every intermediate time is an integer.

Proof. The input consists of m identical rounds; round j occupies $[T_j, T_{j+1}) = [jL, (j+1)L)$. At time T_j inject, in rank order, $k-1$ long jobs of service L and then L short jobs of service 1, so that the long jobs carry the lower ranks. Let P be the static-priority policy that dispatches a waiting short job whenever one exists and a long job otherwise; it is work-conserving.

We show by induction that at T_j first-come first-served has completed everything and idles, while P has exactly $k-1$ jobs in service with total remaining ρ_j , where $\rho_0 = 0$.

First-come first-served in round j . All k servers are free at T_j , so it dispatches the $k-1$ long jobs and one short job. Its remaining server takes the L short jobs back to back and finishes at $T_j + L$; the long jobs also finish at $T_j + L$. So all k servers are busy on (T_j, T_{j+1}) and the system is empty and idle again at T_{j+1} .

P in round j . At T_j , P has one free server, which takes a short job. Its $k-1$ carried-over jobs all complete at $T_j + \rho_j / (k-1)$ and their servers take short jobs as well, and P never touches a long job while a short one waits. So all k servers are busy from T_j until the short

jobs run out, at $T_j + \tau_j$ with $k\tau_j = \rho_j + L$. (P is indeed busy throughout: the carried-over jobs end at $\rho_j/(k-1) \leq \tau_j$, which is $k\rho_j \leq (k-1)(\rho_j + L)$, that is $\rho_j \leq (k-1)L$, true by induction.) At $T_j + \tau_j$ the policy P holds only the $k-1$ long jobs of the round; it dispatches them and leaves one server idle.

The gain. On $(T_j + \tau_j, T_{j+1}]$, first-come first-served has k busy servers and P has $k-1$, so the gap grows at rate 1 over a window of length $L - \tau_j$. At T_{j+1} the reference system is empty, and each of P 's $k-1$ long jobs has run for $L - \tau_j$ and has τ_j left. The hypothesis is reproduced with $\rho_{j+1} = (k-1)\tau_j = (k-1)(\rho_j + L)/k$, whose solution with $\rho_0 = 0$ is $\rho_j = (k-1)L(1 - ((k-1)/k)^j)$. With $L = k^m$ every ρ_j and τ_j for $j \leq m$ is an integer. $\square$

Appendix A.3. Tightness of the net-overtake identity

Proof of Theorem 2. *Upward.* Run the cascade of Lemma A1 for m rounds and, at T_m , inject in rank order $k-1$ jobs of service L , then the victim i of service L , then $N > 2kL$ jobs of service 1. Let P continue as the static priority “the $k-1$ new long jobs first, then the short jobs, then i ”.

At T_m all k servers of the reference policy are free and its waiting set in rank order is the $k-1$ new long jobs, then i , then the short ones. It dispatches the long jobs and then i , all in the phase at T_m . Hence $W_{\text{FCFS}}[i] = 0$; the $k-1$ new long jobs are dispatched strictly before i in its sequence and are in service at $s_i^{\text{FCFS}} = T_m$ with their full service, so $\rho_i^{\text{FCFS}} = (k-1)L$ and $R_i^{\text{FCFS}} = (k-1)L$.

Under P , one server is free at T_m and takes a new long job; when the $k-1$ carried-over jobs finish, all at $T_m + f$ with $f < L$, their servers take the remaining new long jobs and then short jobs. The victim waits while any short job waits. Short jobs are executed at rate at most k , so they are not exhausted before $T_m + N/k$, and $N > 2kL > k(f+L)$ makes that later than $T_m + f + L$, by which time every new long job has completed. So i is dispatched after all $k-1$ new long jobs have finished, and at s_i^P the other $k-1$ servers hold short jobs, giving $0 \leq \rho_i^P \leq k-1$. Also $R_i^P = \rho_m + (k-1)L$, since the carried-over jobs and the new long jobs all have rank below i and the new long jobs are untouched at a_i . Therefore $R_i^P - R_i^{\text{FCFS}} = \rho_m$ and

$$\begin{aligned} \Delta_i &= (R_i^P - R_i^{\text{FCFS}}) - \rho_i^P + \rho_i^{\text{FCFS}} \geq \rho_m + (k-1)L - (k-1) \\ &= 2(k-1)L - (k-1)L\left(\frac{k-1}{k}\right)^m - (k-1), \end{aligned}$$

and both error terms are $o(L)$ as $m \rightarrow \infty$ with $L = k^m$.

Downward. Run the mirrored cascade: inject the L short jobs first in rank order and the $k-1$ long jobs after, and let P be “long jobs before short ones”. The induction of Lemma A1 runs with the roles of the two policies exchanged, so at T_m the policy P is empty and idle while the reference holds $k-1$ jobs with f of service left each. At T_m inject, in rank order, one *filler* job of service f , the victim i of service $L-1$, and $k-1$ jobs of service L ; let P be “the $k-1$ new long jobs first, then i , then the filler”.

At T_m the reference policy has $k-1$ servers busy with carried-over jobs, each with exactly f left, and one free server, which takes the filler — the lowest-ranked waiting job — which also has service f . So all k of its servers complete simultaneously at $T_m + f$, and it then dispatches i , with nothing else dispatched in that phase before it. Hence $W_{\text{FCFS}}[i] = f$ and $\rho_i^{\text{FCFS}} = 0$.

Under P the system is empty at T_m with k free servers, and it dispatches the $k - 1$ new long jobs and then i , all at T_m . So $W_P[i] = 0$, $\text{In}_i = (k - 1)L$, $\rho_i^P = (k - 1)L$, and the filler, of rank below i , is dispatched after i , so $\text{Out}_i = f$. Therefore

$$\begin{aligned}\Delta_i &= k(0 - f) - ((k - 1)L - f) = -(k - 1)f - (k - 1)L \\ &= -\rho_m - (k - 1)L \rightarrow -2(k - 1)L.\end{aligned}$$

Non-attainment. Lemma 1 gives $|R_i^P - R_i^{\text{FCFS}}| < (k - 1)L$ strictly, and step (ii) of Theorem 1 gives $0 \leq \rho_i^P, \rho_i^{\text{FCFS}} \leq (k - 1)L$; combining them in each direction gives $|\Delta_i| < 2(k - 1)L$. $\square$

For the record, the exact values of Δ_i/L produced by these two families, against the ceiling $2(k - 1)$:

k	L	m	upward Δ_i/L	downward Δ_i/L	$2(k - 1)$
2	64	6	$127/64 = 1.984$	$-127/64 = -1.984$	2
3	81	4	$290/81 = 3.580$	$-292/81 = -3.605$	4
4	64	3	$303/64 = 4.734$	$-303/64 = -4.734$	6

In the executed-work convention of Remark 1 the upward column is unchanged to within $o(L)$, while the downward column becomes $-63/64$, $-130/81$ and $-111/64$, that is $-(k - 1)L$ rather than $-2(k - 1)L$. The same-phase share of $|\Delta_i|$ is 0.00%, 0.69% and 0.00% upward and 50.39%, 55.48% and 63.37% downward, and the victim's own excess is $+3.00L$, $+3.20L$, $+3.19L$ upward and $-0.98L$, $-0.80L$, $-0.58L$ downward.

Appendix A.4. Tightness of the guard bound

Proof of Theorem 5(i). At $t = 0$ let there arrive, in rank order, k jobs of service L , the victim i , and at least $B + k$ jobs of service 1; let the base policy be the static priority “the k long jobs, then the short ones, then i ”, and take the constant budget B . First-come first-served dispatches the k long jobs in the phase at $t = 0$, filling every server, so $W_{\text{FCFS}}[i] = L$.

If $B = 0$ then $E(t)$ contains every waiting job at every dispatch epoch and the wrapper is first-come first-served, so the excess is $0 = \lceil B/k \rceil$. If $B > 0$ then at $t = 0$ no job has completed, $E(0)$ is empty, and the base fills the servers with the k long jobs. They complete at $t = L$, and $\text{over}[i](L) = 0$ still, every completed job having rank below i . From $t = L$ on, all k servers are free at each integer instant; while $\text{over}[i] < B$ the set E is empty — the waiting jobs of rank above i have no completed job of still higher rank ahead of them — so the base dispatches k short jobs, which complete one time unit later. Hence $\text{over}[i](L + r) = rk$, the first instant at which $\text{over}[i] \geq B$ is $L + \lceil B/k \rceil$, and there i is the minimum-rank member of E and is dispatched. So $\text{excess}_P[i] = \lceil B/k \rceil$. $\square$

Proof of Theorem 5(ii). The upper bound $c(k) < 3k - 2$ is Theorem 3: for a constant budget, $k \text{ excess}_P[i] \leq \text{In}_i - \text{Out}_i + 2(k - 1)L \leq \text{In}_i + 2(k - 1)L < B + kL + 2(k - 1)L = B + (3k - 2)L$, the last step being the strict inequality (3). When i has no overtaker the excess is at most $(2 - 2/k)L$ and the inequality is strict as well.

For the lower bound, fix $k \geq 2$, $m \geq 1$, $L = k^m$, and let f and ρ_m be as in Lemma A1. Run the cascade for m rounds. At T_m inject, all arriving at T_m and in rank order,

- $k - 1$ new long jobs of service L , of rank below the victim;
- the victim i , of service L ;
- one *sync* job of service f ;
- one *pre* job of service L ;
- k final jobs of service L ;

set the constant budget to $B = f + L + 1$, and let the base policy be the static priority “short jobs of the cascade, then sync, then the new long jobs and pre, then the finals, and the victim last”.

The reference run. At T_m it is empty with all k servers free; it dispatches the $k - 1$ new long jobs and then i in the phase at T_m , so $W_{\text{FCFS}}[i] = 0$.

The guard does not fire during the cascade. Inside round j the only waiting jobs with completed higher-ranked work are that round’s long jobs, whose higher-ranked jobs are the L short jobs of the round and everything in later rounds, which have not arrived. Over $[T_j, T_j + \tau_j]$ the k servers execute $k\tau_j = \rho_j + L$ units of work, of which ρ_j belongs to the carried-over jobs, so at most L units of short-job work can have completed. Hence $\text{over}[q](t) \leq L < B$ for every waiting q at every dispatch epoch of the cascade, $E(t)$ is empty throughout, and the wrapped run reproduces Lemma A1.

The finale. At T_m one server is free and $\text{over}[q](T_m) = 0$ for every waiting q , so the base chooses and takes the sync job. The $k - 1$ carried-over jobs have exactly f left and the sync job has service f , so all k servers free simultaneously at $T_m + f$. There $\text{over}[i] = f$, because only the sync job has rank above i among the completed jobs, and $f < B$; no other waiting job has a completed higher-ranked job either, so E is empty and the base fills all k servers with the $k - 1$ new long jobs and pre. These all have service L and complete simultaneously at $T_m + f + L$, where

$$\text{over}[i] = f + L = B - 1 < B,$$

since only sync and pre have rank above i among the completed jobs. So E is still empty, the base fills all k servers with the k final jobs, and they complete at $T_m + f + 2L$, at which instant $\text{over}[i] = B - 1 + kL \geq B$. The guard fires, i is the minimum-rank member of E , and it is dispatched with every server free, so $\rho_i^P = 0$.

The value. $W_P[i] = f + 2L$, $\text{In}_i = B - 1 + kL$, $\text{Out}_i = 0$, and

$$\frac{k \text{excess}_P[i] - B}{L} = \frac{k(f + 2L) - (f + L + 1)}{L} = \frac{(k - 1)f + (2k - 1)L - 1}{L}.$$

As $m \rightarrow \infty$ with $L = k^m$ we have $f/L \rightarrow 1$ and $1/L \rightarrow 0$, so the ratio tends to $(k - 1) + (2k - 1) = 3k - 2$.

The same instance also has $\Delta_i = \rho_m + (k - 1)L \rightarrow 2(k - 1)L$, so the two sources of slack in Theorem 3 are saturated together. $\square$

The values produced, against the ceiling $3k - 2$:

k	L	m	jobs	B	excess	$(k \text{excess} - B)/L$
2	64	6	396	128	191	$127/32 = 3.969$
2	128	7	909	256	383	$255/64 = 3.984$
3	81	4	340	147	227	$178/27 = 6.593$
3	243	5	1233	455	697	$1636/243 = 6.733$
4	64	3	211	102	165	$279/32 = 8.719$
4	256	4	1046	432	687	$579/64 = 9.047$
5	125	3	399	187	311	$1368/125 = 10.944$
6	216	3	677	308	523	$1415/108 = 13.102$

Appendix A.5. Consistency, and why the converse fails

Proof of Proposition 1. Induct on dispatch epochs: up to the first epoch at which the two runs could differ their completion histories agree, hence so does $\text{over}[\cdot]$, so $E(t)$ is empty in

the wrapped run and it copies A . The first consequence uses $\text{over}[q](t) \leq \text{In}_q^A$, a completed overtaker of q having been dispatched while q waited; the second uses Corollary 2. $\square$

The converse fails, so this is not an equivalence. Take $k = 1$, two jobs of service 1 at $t = 0$, the base policy “higher rank first”, and $B = 1$. Both runs dispatch in the order $(1, 0)$, yet at the second dispatch the waiting job has $\text{over} = 1 \geq B$, so the guard fires and happens to choose what the base would have chosen. Nor is there multiplicative consistency: with $k = 1$, one job of service L followed by m jobs of service 1 all at $t = 0$, the wrapper with $B = 0$ is first-come first-served while the base ordering by service time is not, and the ratio of mean waits is $(2L + m - 1)/(m + 1)$, unbounded in L .

Appendix A.6. Approaching the single-server bound

Proof of the second claim of Proposition 3. Fix L , a budget B , and a granularity g dividing B with $0 < g \leq L$. Take $k = 1$ and, at $t = 0$, the victim i of rank 0, then $B/g - 1$ jobs of service g , then one job of service L ; let the base policy dispatch the jobs of service g first, then the one of service L , and never the victim. First-come first-served starts i at once, so $W_{\text{FCFS}}[i] = 0$.

Under the wrapper, after r of the jobs of service g have completed, $\text{over}[i] = rg$, which stays below B for $r \leq B/g - 1$. At the last such instant $\text{over}[i] = B - g < B$, the guard does not fire, and the base dispatches the job of service L . When it completes, $\text{over}[i] = B - g + L \geq B$ and the guard releases i . Hence $\text{excess}_P[i] = B - g + L$, so $\text{excess}_P[i] - B = L - g$, which approaches L as $g \rightarrow 0$ and never reaches it. For example $(L, B, g) = (100, 40, 1)$ gives $\text{excess}_P[i] - B = 99$, and $(64, 32, 1)$ gives 63. $\square$

Appendix A.7. The price of the guarantee

Proof of Theorem 6. Let $S(Q)$ be the set of inverted pairs of a policy Q , that is the pairs (u, v) with $u < v$ in rank and $v \prec u$. Counting the same pairs in two ways,

$$\sum_i \text{In}_i = \sum_{(u,v) \in S(Q)} C_v, \quad \sum_i \text{Out}_i = \sum_{(u,v) \in S(Q)} C_u, \quad (A1)$$

exactly, for every k . Write $\Delta(Q) = \sum_i (W_{\text{FCFS}}[i] - W_Q[i])$ for the total saving against the reference and $E = 2(1 - 1/k)L(m + n)$. Summing Theorem 1 over the $m + n$ jobs gives

$$\left| \Delta(Q) - \frac{1}{k} \sum_{(u,v) \in S(Q)} (C_u - C_v) \right| \leq E \quad \text{for every } Q. \quad (A1)$$

Numerator. A pair contributes positively only when u is a long job and v a short one, contributing $L - s$. For a fixed long job u , the number of short jobs dispatched before it is at most In_u/s , and Corollary 2 gives $\text{In}_u \leq kG + (3k - 2)L$. Summing over the m long jobs and applying (A1),

$$\Delta(P) \leq \frac{1}{k} m(L - s) \frac{kG + (3k - 2)L}{s} + E.$$

Denominator. Sorting by service time inverts every one of the mn long-short pairs, so its pair sum is exactly $mn(L - s)$, and (A1) applied to it gives $\Delta(\text{SJF}) \geq \frac{1}{k} mn(L - s) - E$. At $k > 1$ the denominator carries the same error as the numerator, so treating the reference saving as exact would understate ϵ .

Division. Both bounds are positive once $mn(L - s)/k > E$, and then

$$\frac{\Delta(P)}{\Delta(\text{SJF})} \leq \frac{\frac{1}{k} m(L - s) (kG + (3k - 2)L)/s + E}{\frac{1}{k} mn(L - s) - E} = \frac{kG + (3k - 2)L}{ns} + \epsilon,$$

where ε collects both corrections. The leading terms are $\Theta(mn)$ and $E = \Theta(m + n)$, so $\varepsilon = O((m + n)/(mn)) \rightarrow 0$. Dividing numerator and denominator by $m + n$ turns the two total savings into the two mean savings and leaves the ratio unchanged. $\square$

At $k = 1$ the error term vanishes identically, the sum identity is exact, and the achievable fraction is $\min(1, G/(ns))$: the only way to beat first-come first-served in mean wait is to invert pairs in which the later-ranked job is the shorter one, and the saving is exactly the sum of the service-time differences over the inversions.

References

1. Wasik, S.; Antczak, M.; Badura, J.; Laskowski, A.; Sternal, T. A Survey on Online Judge Systems and Their Applications. *ACM Computing Surveys* **2018**, *51*, 1–34. <https://doi.org/10.1145/3143560>.
2. DOMjudge team. DOMjudge Administrator's Manual: Overview and Judging. <https://www.domjudge.org/docs/manual/main/overview.html>. Accessed 2026-09-19.
3. Peveler, M.; Maicus, E.; Cutler, B. Comparing Jailed Sandboxes vs Containers Within an Autograding System. In Proceedings of the Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE '19), 2019, pp. 139–145. <https://doi.org/10.1145/3287324.3287507>.
4. König, C.J.; Kleinmann, M. Deadline Rush: A Time Management Phenomenon and Its Mathematical Description. *The Journal of Psychology* **2005**, *139*, 33–45. <https://doi.org/10.3200/JRLP.139.1.33-45>.
5. Alfi, V.; Parisi, G.; Pietronero, L. Conference registration: how people react to a deadline. *Nature Physics* **2007**, *3*, 746. <https://doi.org/10.1038/nphys761>.
6. Alfi, V.; Gabrielli, A.; Pietronero, L. How people react to a deadline: time distribution of conference registrations and fee payments. *Open Physics* **2009**, *7*, 483–489. <https://doi.org/10.2478/s11534-009-0059-z>.
7. Microsoft Azure. Azure Functions Invocation Trace 2021. <https://github.com/Azure/AzurePublicDataset/blob/master/AzureFunctionsInvocationTrace2021.md>, 2021.
8. Zhang, Y.; Goiri, Í.; Chaudhry, G.I.; Fonseca, R.; Elnikety, S.; Delimitrou, C.; Bianchini, R. Faster and Cheaper Serverless Computing on Harvested Resources. In Proceedings of the Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21), 2021, pp. 724–739. <https://doi.org/10.1145/3477132.3483580>.
9. Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; Liu, T.Y. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In Proceedings of the Advances in Neural Information Processing Systems 30 (NeurIPS 2017), 2017.
10. Schlichtkrull, M.; Kipf, T.N.; Bloem, P.; van den Berg, R.; Titov, I.; Welling, M. Modeling Relational Data with Graph Convolutional Networks. In Proceedings of the The Semantic Web (ESWC 2018), 2018, Vol. 10843, *Lecture Notes in Computer Science*, pp. 593–607. Preprint arXiv:1703.06103, https://doi.org/10.1007/978-3-319-93417-4_38.
11. Scully, Z.; Grosz, I.; Mitzenmacher, M. Uniform Bounds for Scheduling with Job Size Estimates. In Proceedings of the 13th Innovations in Theoretical Computer Science Conference (ITCS 2022). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, Vol. 215, *LIPICs*, pp. 114:1–114:30. Preprint arXiv:2110.00633, <https://doi.org/10.4230/LIPICs.ITCS.2022.114>.
12. Mitzenmacher, M.; Shahout, R. Queueing, Predictions, and Large Language Models: Challenges and Open Problems. *Stochastic Systems* **2025**. Preprint arXiv:2503.07545, <https://doi.org/10.1287/stsy.2025.0106>.
13. Universidade Federal do Amazonas, Instituto de Computação. CodeBench Dataset. <https://codebench.icomp.ufam.edu.br/dataset/>. Accessed 2026-09-19.
14. Chen, K.; Huang, F.; Liu, Z.; Yu, H.; Meng, L.; Mo, S.; Zhang, L.; Song, Y. ACcoding: A graph-based dataset for online judge programming. *Scientific Data* **2024**, *11*. See also Author Correction, doi:10.1038/s41597-024-03693-3, <https://doi.org/10.1038/s41597-024-03392-z>.
15. Parallel Workloads Archive. The Intel Netbatch Logs. https://www.cs.huji.ac.il/labs/parallel/workload/I_intel_netbatch/, 2012.
16. Shai, O.; Shmueli, E.; Fetselson, D.G. Heuristics for Resource Matching in Intel's Compute Farm. In Proceedings of the Job Scheduling Strategies for Parallel Processing (JSSPP 2013). Springer, 2014, *Lecture Notes in Computer Science*, pp. 116–135. https://doi.org/10.1007/978-3-662-43779-7_7.
17. Purohit, M.; Svitkina, Z.; Kumar, R. Improving Online Algorithms via ML Predictions. In Proceedings of the Advances in Neural Information Processing Systems 31 (NeurIPS 2018), 2018.
18. Mitzenmacher, M. Scheduling with Predictions and the Price of Misprediction. In Proceedings of the 11th Innovations in Theoretical Computer Science Conference (ITCS 2020). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, Vol. 151, *LIPICs*, pp. 14:1–14:18. Preprint arXiv:1902.00732, <https://doi.org/10.4230/LIPICs.ITCS.2020.14>.

19. Azar, Y.; Leonardi, S.; Touitou, N. Flow Time Scheduling with Uncertain Processing Time. In Proceedings of the Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC), 2021, pp. 1070–1080.
20. Azar, Y.; Leonardi, S.; Touitou, N. Distortion-Oblivious Algorithms for Minimizing Flow Time. In Proceedings of the Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 2022, pp. 252–274. Preprint arXiv:2109.08424.
21. Dong, J.; Ibrahim, R. Shortest-Job-First Scheduling in Many-Server Queues with Impatient Customers and Noisy Service-Time Estimates. *Operations Research* **2025**, *73*, 3139–3155. <https://doi.org/10.1287/opre.2022.310>.
22. Shahout, R.; Mitzenmacher, M. SkipPredict: When to Invest in Predictions for Scheduling. In Proceedings of the Advances in Neural Information Processing Systems 37 (NeurIPS 2024), 2024, pp. 105663–105701. Preprint arXiv:2402.03564, <https://doi.org/10.52202/079017-3352>.
23. Sabin, G.; Sadayappan, P. Unfairness Metrics for Space-Sharing Parallel Job Schedulers. In Proceedings of the Job Scheduling Strategies for Parallel Processing (JSSPP 2005), 2005, Vol. 3834, *Lecture Notes in Computer Science*, pp. 238–256. https://doi.org/10.1007/11605300_12.
24. Mu'alem, A.W.; Feitelson, D.G. Utilization, predictability, workloads, and user runtime estimates in scheduling the IBM SP2 with backfilling. *IEEE Transactions on Parallel and Distributed Systems* **2001**, *12*, 529–543. <https://doi.org/10.1109/71.932708>.
25. Lindsay, A.M.; Galloway-Carson, M.; Johnson, C.R.; Bunde, D.P.; Leung, V.J. Backfilling with guarantees made as jobs arrive. *Concurrency and Computation: Practice and Experience* **2013**, *25*, 513–523. <https://doi.org/10.1002/cpe.2860>.
26. Grosf, I.; Yang, K.; Scully, Z.; Harchol-Balter, M. Nudge: Stochastically Improving upon FCFS. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2021**, *5*, 1–29. Preprint arXiv:2106.01492, <https://doi.org/10.1145/3460088>.
27. Van Houdt, B. On the Stochastic and Asymptotic Improvement of First-Come First-Served and Nudge Scheduling. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2022**, *6*, 1–22. Preprint arXiv:2206.10428, <https://doi.org/10.1145/3570610>.
28. Charlet, N.; Van Houdt, B. Tail Optimality and Performance Analysis of the Nudge*(M) Scheduling Algorithm. *Operations Research* **2026**, *74*, 2161–2178. <https://doi.org/10.1287/opre.2024.1061>.
29. Grosf, I.; Harchol-Balter, M.; Scheller-Wolf, A. WCFS: a new framework for analyzing multiserver systems. *Queueing Systems* **2022**, *102*, 143–174. Preprint arXiv:2109.12663, <https://doi.org/10.1007/s11134-022-09848-6>.
30. Dai, J.G.; Deng, T.; Li, Y.; Peng, T. Throughput-Optimal Scheduling Algorithms for LLM Inference and AI Agents, 2025, [2504.07347].
31. Parekh, A.K.; Gallager, R.G. A generalized processor sharing approach to flow control in integrated services networks: the single-node case. *IEEE/ACM Transactions on Networking* **1993**, *1*, 344–357. <https://doi.org/10.1109/90.234856>.
32. Parekh, A.K.; Gallager, R.G. A generalized processor sharing approach to flow control in integrated services networks: the multiple node case. *IEEE/ACM Transactions on Networking* **1994**, *2*, 137–150. <https://doi.org/10.1109/90.298432>.
33. Kleinrock, L. A delay dependent queue discipline. *Naval Research Logistics Quarterly* **1964**, *11*, 329–341. <https://doi.org/10.1002/nav.3800110306>.
34. Stanford, D.A.; Taylor, P.; Ziedins, I. Waiting time distributions in the accumulating priority queue. *Queueing Systems* **2014**, *77*, 297–330. Online first 2013-12-07, <https://doi.org/10.1007/s11134-013-9382-6>.
35. Bansal, N.; Harchol-Balter, M. Analysis of SRPT scheduling. In Proceedings of the Proceedings of the 2001 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2001, pp. 279–290. <https://doi.org/10.1145/378420.378792>.
36. Wierman, A.; Harchol-Balter, M. Classifying scheduling policies with respect to unfairness in an M/GI/1. In Proceedings of the Proceedings of the 2003 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems, 2003, pp. 238–249. <https://doi.org/10.1145/781027.781057>.
37. Kiefer, J.; Wolfowitz, J. On the theory of queues with many servers. *Transactions of the American Mathematical Society* **1955**, *78*, 1–18. <https://doi.org/10.2307/1992945>.
38. Daley, D.J. Certain optimality properties of the first-come first-served discipline for G/G/s queues. *Stochastic Processes and their Applications* **1987**, *25*, 301–308. [https://doi.org/10.1016/0304-4149\(87\)90208-0](https://doi.org/10.1016/0304-4149(87)90208-0).
39. Tsafir, D.; Etsion, Y.; Feitelson, D.G. Backfilling Using System-Generated Predictions Rather than User Runtime Estimates. *IEEE Transactions on Parallel and Distributed Systems* **2007**, *18*, 789–803. <https://doi.org/10.1109/TPDS.2007.70606>.
40. Gaussier, E.; Glessner, D.; Reis, V.; Trystram, D. Improving backfilling by using machine learning to predict running times. In Proceedings of the Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15), 2015, pp. 1–10. <https://doi.org/10.1145/2807591.2807646>.
41. Gaussier, E.; Lelong, J.; Reis, V.; Trystram, D. Online Tuning of EASY-Backfilling using Queue Reordering Policies. *IEEE Transactions on Parallel and Distributed Systems* **2018**, *29*, 2304–2316. <https://doi.org/10.1109/TPDS.2018.2820699>.
42. Zhang, E.; Wu, F.; Lu, X. An Intelligent Scheduling System for Large-Scale Online Judging. In Proceedings of the Computer Science and Education. Computer Science and Technology (ICCSE 2023). Springer Nature Singapore, 2024, Communications in Computer and Information Science, pp. 265–279. https://doi.org/10.1007/978-981-97-0730-0_24.

43. Fu, Y.; Zhu, S.; Su, R.; Qiao, A.; Stoica, I.; Zhang, H. Efficient LLM Scheduling by Learning to Rank, 2024, [2408.15792]. 1644
44. Li, Y.; Chen, Y.; Chen, J.; et al. Beyond Prediction: Tail-Aware Scheduling for LLM Inference, 2026, [2606.18431]. 1645
45. Luo, Z.; Liu, J.; Lee, M.; Shroff, N.B. Prediction-Assisted Online Distributed Deep Learning Workload Scheduling in GPU Clusters. In Proceedings of the IEEE INFOCOM 2025 – IEEE Conference on Computer Communications, 2025, pp. 1–10. Preprint arXiv:2501.05563, <https://doi.org/10.1109/INFOCOM55648.2025.11044761>. 1646
46. Massey, W.A.; Whitt, W. Peak congestion in multi-server service systems with slowly varying arrival rates. *Queueing Systems* **1997**, *25*, 157–172. <https://doi.org/10.1023/A:1019156418862>. 1649
47. Jennings, O.B.; Mandelbaum, A.; Massey, W.A.; Whitt, W. Server Staffing to Meet Time-Varying Demand. *Management Science* **1996**, *42*, 1383–1394. <https://doi.org/10.1287/mnsc.42.10.1383>. 1651
48. Green, L.V.; Kolesar, P.J.; Whitt, W. Coping with Time-Varying Demand When Setting Staffing Requirements for a Service System. *Production and Operations Management* **2007**, *16*, 13–39. <https://doi.org/10.1111/j.1937-5956.2007.tb00164.x>. 1653
49. Feldman, Z.; Mandelbaum, A.; Massey, W.A.; Whitt, W. Staffing of Time-Varying Queues to Achieve Time-Stable Performance. *Management Science* **2008**, *54*, 324–338. <https://doi.org/10.1287/mnsc.1070.0821>. 1655
50. Harchol-Balter, M. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*; Cambridge University Press, 2013. <https://doi.org/10.1017/CBO9781139226424>. 1657
51. Xu, K.; Hu, W.; Leskovec, J.; Jegelka, S. How Powerful are Graph Neural Networks? In Proceedings of the International Conference on Learning Representations (ICLR 2019), 2019. Preprint arXiv:1810.00826. 1659
52. Robinson, J.; Ranjan, R.; Hu, W.; Huang, K.; Han, J.; Dobles, A.; Fey, M.; Lenssen, J.E.; Yuan, Y.; Zhang, Z.; et al. RelBench: A Benchmark for Deep Learning on Relational Databases. In Proceedings of the Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track (NeurIPS 2024), 2024. Preprint arXiv:2407.20060. 1661
53. Gu, J.; Ranjan, R.; Kanatsoulis, C.; Tang, H.; Jurkovic, M.; Hudovernik, V.; Znidar, M.; Chaturvedi, P.; Shroff, P.; Li, F.; et al. RelBench v2: A Large-Scale Benchmark and Repository for Relational Data, 2026, [2602.12606]. 1664
54. Grinsztajn, L.; Oyallon, E.; Varoquaux, G. Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data? In Proceedings of the Advances in Neural Information Processing Systems 35 (NeurIPS 2022), 2022, pp. 507–520. <https://doi.org/10.52202/068431-0037>. 1666
55. Jørgensen, B. Exponential Dispersion Models. *Journal of the Royal Statistical Society, Series B* **1987**, *49*, 127–145. <https://doi.org/10.1111/j.2517-6161.1987.tb01685.x>. 1668
56. Yang, Y.; Qian, W.; Zou, H. Insurance Premium Prediction via Gradient Tree-Boosted Tweedie Compound Poisson Models. *Journal of Business & Economic Statistics* **2018**, *36*, 456–470. <https://doi.org/10.1080/07350015.2016.1200981>. 1669
57. Mozilla. Taskcluster Documentation: Queue Service Reference. <https://docs.taskcluster.net/docs/reference/platform/queue>. Accessed 2026-09-19. 1671
58. Mozilla. Treeherder REST API Documentation. <https://treeherder.mozilla.org/docs/>. Accessed 2026-09-19. 1672
59. Kuzilek, J.; Hlosta, M.; Zdrahal, Z. Open University Learning Analytics dataset. *Scientific Data* **2017**, *4*, 170171. <https://doi.org/10.1038/sdata.2017.171>. 1673
60. Shahradd, M.; Fonseca, R.; Goiri, Í.; Chaudhry, G.; Batur, P.; Cooke, J.; Laureano, E.; Tresness, C.; Russinovich, M.; Bianchini, R. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC 20). USENIX Association, 2020, pp. 205–218. 1674
61. Verma, A.; Pedrosa, L.; Korupolu, M.; Oppenheimer, D.; Tune, E.; Wilkes, J. Large-scale cluster management at Google with Borg. In Proceedings of the Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15), 2015, pp. 1–17. <https://doi.org/10.1145/2741948.2741964>. 1675
62. Smith, D.R.; Whitt, W. Resource Sharing for Efficiency in Traffic Systems. *Bell System Technical Journal* **1981**, *60*, 39–55. <https://doi.org/10.1002/j.1538-7305.1981.tb00221.x>. 1676
63. Feitelson, D.G. Resampling with Feedback: A New Paradigm of Using Workload Data for Performance Evaluation. In *Job Scheduling Strategies for Parallel Processing (JSSPP 2021)*; Lecture Notes in Computer Science, Springer, 2021; pp. 3–32. https://doi.org/10.1007/978-3-030-88224-2_1. 1677
64. Shmueli, E.; Feitelson, D.G. On Simulation and Design of Parallel-Systems Schedulers: Are We Doing the Right Thing? *IEEE Transactions on Parallel and Distributed Systems* **2009**, *20*, 983–996. <https://doi.org/10.1109/TPDS.2008.152>. 1678
65. Harchol-Balter, M.; Schroeder, B.; Bansal, N.; Agrawal, M. Size-based scheduling to improve web performance. *ACM Transactions on Computer Systems* **2003**, *21*, 207–233. <https://doi.org/10.1145/762483.762486>. 1679
66. Harchol-Balter, M. Task assignment with unknown duration. *Journal of the ACM* **2002**, *49*, 260–288. <https://doi.org/10.1145/506147.506154>. 1680

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content. 1695